TFL-TAGE-SC-L: A TAGE-SC-L Branch Predictor with Target-Free Loop-Structure History

Evaluation on 50 SPEC CPU2017 traces against the ChampSim default predictors

November 2026

Abstract

TFL-TAGE-SC-L is a TAGE-SC-L branch predictor for the 2021 ChampSim interface, extended with loop-structure history (IMLI-SIC, IMHIST and a backward-branch history) that is made usable by a target-free inference of backward branches. The frozen design was evaluated on 50 SPEC CPU2017 traces (the 10 traces of the earlier study and 40 additional ones) against all four predictors shipped with ChampSim (bimodal, gshare, perceptron, hashed perceptron) and against our own TAGE and TAGE-SC-L. Every predictor ran on every trace with 50M warm-up and 200M measured instructions (350 simulations, 350 successful).

TFL-TAGE-SC-L has the highest geometric-mean IPC (0.8914), the highest pooled accuracy (98.795%) and the lowest mean MPKI (1.628) of the seven predictors. Against the strongest ChampSim predictor, the 64 KB hashed perceptron, it removes 15.29% of the mispredictions per kilo-instruction and improves geometric-mean IPC by +0.772%, with fewer mispredictions on 46 of 50 traces, while using 15,876 fewer bits. Against our TAGE-SC-L without the loop-structure components, the gain is small: 2.09% lower mean MPKI and +0.052% geometric-mean IPC for 6.8% more storage, with fewer mispredictions on 29 traces and more on 16. 43.2% of that MPKI reduction comes from a single trace. The improvement observed on the original ten traces carries over to the 40 traces never used during design: there the MPKI reduction is 19.93% relative to the hashed perceptron and 2.82% relative to TAGE-SC-L.

Contents

1 Introduction and objective **3**

2 Background and related work **4**

3 Predictor implementation **4**
3.1 Baselines 4
3.2 Architecture of TFL-TAGE-SC-L 5
3.3 Prediction and training workflow 6
3.4 Implementation and integration 6

4 Experimental methodology **7**
4.1 Simulator configuration 7
4.2 Benchmark trace description 7
4.3 Metrics 7
4.4 Validity checks 8

5 Hardware and storage accounting **8**

6 Results **9**
6.1 Overall comparison 9
6.2 TFL-TAGE-SC-L relative to each baseline 11
6.3 Per-trace analysis 13
6.4 Generalisation: original 10 versus additional 40 traces 16
6.5 Storage versus performance 18
6.6 MPKI reduction versus IPC gain 19
6.7 Simulation runtime 20

7 Discussion **20**

8 Relation to the earlier 10-trace study **21**

9 Limitations and threats to validity **21**

10 Conclusion and future work **22**

A IPC, MPKI and accuracy of every predictor on every trace **23**

B Trace manifest **26**

C Reproduction **27**

1 Introduction and objective

The assignment asks for a branch predictor implemented in ChampSim that improves on suitable baselines, evaluated with IPC, branch prediction accuracy and MPKI. We use the ChampSim version supplied for the course (2021, `build_champsim.sh`), whose predictor interface is

```
uint8_t O3_CPU::predict_branch(uint64_t ip);
void    O3_CPU::last_branch_result(uint64_t ip, uint8_t taken);
```

Both functions are called in `O3_CPU::read_from_trace()` for every instruction that the trace marks as a branch, in trace order, and `last_branch_result()` immediately follows `predict_branch()` with the true outcome. The predictor is never told the branch type or the target, and there is no wrong-path fetch. A misprediction stalls fetch until the branch executes.

Problem statement. Design a branch predictor for this ChampSim interface that improves on the predictors shipped with ChampSim in IPC, accuracy and MPKI, at a storage budget comparable to the largest of them (the 64 KB hashed perceptron), and demonstrate the improvement experimentally.

Motivation. Among the predictors shipped with this ChampSim the hashed perceptron (64 KB) is by far the most accurate, and in the design phase (earlier study) a TAGE of similar storage was not better on average because it lost on traces dominated by data-dependent, hard-to-predict branches. Combining TAGE with a perceptron-like statistical corrector (TAGE-SC-L) is the established remedy [11, 12]. Part of TAGE-SC-L's corrector accuracy comes from *loop-structure history* — the inner-most-loop iteration counter (IMLI) [14], the per-IMLI outcome history (IMHIST) and a history of backward branches — which is defined on backward branches and therefore needs the branch target, information this interface hides. The question addressed is whether those features can be recovered inside the predictor, without changing the simulator, and what they buy.

Proposed predictor and novelty. TFL-TAGE-SC-L is a re-implementation of TAGE-SC-L extended with IMLI-SIC, IMHIST and a backward-branch history. TAGE, TAGE-SC-L, IMLI, IMHIST and backward-branch histories are published techniques. The novelty of this work is a *target-free inference* of backward branches from the order in which branches reach the predictor, which makes these loop-structure components usable in an interface that provides no target, together with their integration and evaluation (Section 3).

Objective of this report. The design and its parameters were fixed and verified in an earlier study on ten traces. This report evaluates the frozen design on a much larger collection: 50 SPEC CPU2017 traces (the original 10 plus 40 additional traces), against every branch predictor shipped with this ChampSim version (bimodal, gshare, perceptron, hashed perceptron) and against our own TAGE and TAGE-SC-L reference configurations, with identical simulation settings for all seven predictors (350 simulations). The questions are: which predictor performs best overall; how TFL-TAGE-SC-L compares with each baseline on IPC, accuracy and MPKI; on how many traces it wins or loses; where the largest gains and regressions occur; what the storage cost is; and whether the result obtained on the original ten traces generalises.

Relation to the earlier 10-trace study. All numbers in this report come from the new 50-trace experiment (stage `M50_50M_200M`). The earlier report and its results are preserved unchanged in `archive/10trace_study/`; Section 8 explains how the two experiments relate. No predictor source, parameter or configuration was changed for this evaluation, and no tuning was performed on the new traces.

2 Background and related work

Table 1: Related work used in this project.

Work	Core idea	Use in this project
Jiménez & Lin, HPCA 2001 [6]	Perceptron per branch over global history	Baseline; basis of the corrector's adder
Seznec & Michaud 2006 [13]; Seznec 2011 [10]	TAGE: tagged tables, geometric history lengths, longest match provides	Base predictor (re-implemented)
Seznec, CBP-4/CBP-5 [11, 12]	TAGE-SC-L: TAGE + statistical corrector + loop predictor	Architecture and update rules followed
Seznec et al., MICRO 2015 [14]	IMLI counter; IMLI-SIC and IMLI-OH corrector components	IMLI-SIC used; IMHIST is the CBP-2016 successor of IMLI-OH
Jiménez, CBP-5 [5]	Multiperspective perceptron: many feature types in one adder	Motivates heterogeneous corrector features
Michaud, TACO 2018 [8]	BATAGE; controlled allocation throttling	Motivated an allocation-throttle experiment (rejected)
Lin & Tarsa, IISWC 2019 [7]	Most mispredictions come from a few hard-to-predict branches	Explains the residual mispredictions on mcf/xz
ChampSim2 PR #2 [3]	TAGE-SC-L adapter that detects backward branches from the simulator's next instruction	Closest related engineering; ours uses only predictor-visible information

Newer work (CBP-2025 entries, LLBP [9], BranchNet, Whisper) needs much larger storage, offline training, ISA hints or data values the traces do not carry; it was reviewed but not used. The simulator is described in [4].

3 Predictor implementation

This section describes the implementation that was verified in the earlier study; it is unchanged.

3.1 Baselines

Table 2: Predictors compared (storage = logical predictor state; see Section 5).

Predictor	Design (from the source)	Storage
Bimodal (ChampSim)	16,384 2-bit counters indexed by PC mod 16381	4.00 KB
Gshare (ChampSim)	16,384 2-bit counters indexed by hash(PC) XOR 14-bit global history	4.00 KB
Perceptron (ChampSim) [6]	163 perceptrons × 25 8-bit weights, 24-bit history, $\theta = 60$	3.98 KB
Hashed perceptron (ChampSim)	16 tables × 4,096 8-bit weights, history lengths 0–232, adaptive threshold	64.03 KB
TAGE (ours)	Bimodal + 12 tagged tables (Section 3.2)	46.84 KB
TAGE-SC-L (ours)	+ 64-entry loop predictor + statistical corrector (bias, global, path, local tables)	58.14 KB
TFL-TAGE-SC-L (proposed)	+ IMLI-SIC, IMHIST, backward-branch history via target-free inference	62.09 KB

The four ChampSim predictors are unchanged. TAGE and TAGE-SC-L are configurations of the same implementation as the proposed design with components switched off at compile time (`-DTSL_NO_SC -DTSL_NO_LOOP` and `-DTSL_NO_IMLI`), so the comparison with them isolates the components

rather than implementation differences. They are not ChampSim defaults and are reported as project reference baselines.

Why these baselines. The four ChampSim predictors are the complete set of branch predictors shipped with this ChampSim version, so the comparison omits none of the simulator's defaults; they also cover the main families (counter tables, global-history hashing, perceptrons) and include the two reference papers' predictors (perceptron [6]; the hashed perceptron as its multi-table successor). The hashed perceptron is the strongest default and has a similar storage budget, so it is the main like-for-like competitor. TAGE [10] is the second reference paper's design, and TAGE-SC-L is the state-of-the-art family the proposal builds on: comparing with our own TAGE-SC-L (the same code without the loop-structure components) isolates the contribution of the novelty from that of the base design.

3.2 Architecture of TFL-TAGE-SC-L

TFL-TAGE-SC-L (TAGE-SC-L with Target-Free Loop-structure history) is implemented in `branch/tagescl.bpred` and selected by `branch/tfl_tage_scl.bpred` (`#define TSL_IMHIST`, `#define TSL_BHIST`).

Target-free backward-branch inference. Let B be a taken branch with target T and N the next branch presented to the predictor. Execution proceeds sequentially from T to N, so $pc(N) \geq T$. If $T \leq pc(B)$, the sequential path from T reaches B itself, so $pc(N) \leq pc(B)$; if $T > pc(B)$ then $pc(N) \geq T > pc(B)$. Hence, for taken instances,

$$B \text{ is backward} \iff pc(\text{next branch presented to the predictor}) \leq pc(B).$$

The test runs at the start of the next `predict_branch()` call, before any later prediction uses the loop state. Two 4,096-bit tables extend it: `bwd` remembers the inferred direction of each (hashed) branch for its not-taken, loop-exit instances, and `cond` marks a branch as conditional once it has been seen not-taken. In the earlier study the inference agreed with ChampSim's own branch targets on 99.9996–100% of taken branches on all ten original traces (50M instructions each).

Table 3: TFL-TAGE-SC-L structures; the total equals the value printed by the predictor's `storage_bits()`.

Structure	Configuration	Bits
Bimodal base	8K × 2-bit counters	16,384
Tagged tables T1–T12	2,048 entries each: 3-bit counter, 1-bit useful, 8–13-bit tag; histories 4, 6, 10, 16, 25, 40, 64, 101, 160, 254, 403, 640	366,592
TAGE control	use_alt_on_na 4 × 5 bit, TICK, 640-bit global + 27-bit path history	698
Loop predictor	64 entries (4-way): 2 × 10-bit trip counts, 10-bit tag, 4-bit confidence, 4-bit age, direction; WITHLOOP	2,503
SC bias tables	3 × 1,024 × 6-bit	18,432
SC global + path GEHL	4 global (lengths 6–40) + 3 path (9–25) tables × 1,024 × 6-bit	43,008
SC control	adaptive threshold, its counter, two chooser counters	30
SC local GEHL	4 × 1,024 × 6-bit (lengths 3–16) + 256 × 16-bit local histories	28,672
IMLI-SIC	512 × 6-bit table + 10-bit IMLI counter	3,082
Backward inference	`bwd` and `cond` tables (4,096 bits each) + previous branch PC and outcome	8,257
IMHIST	256 × 10-bit histories (one per IMLI value) + 1,024 × 6-bit table	8,704
Backward-branch history	16-bit history + 2 × 1,024 × 6-bit tables	12,304
Total		**508,666 (62.09 KB)**

Not counted: the folded-history registers used for hashing (422 bits; recomputable from the global history) and the 64-bit pseudo-random generator state.

3.3 Prediction and training workflow

Prediction.

1. **Loop-structure state.** If the previous branch was taken, classify it as backward or forward from the current PC, store the bit in `bwd`, shift a 1 into the backward history if backward, and increment IMLI if the branch is backward and known to be conditional.
2. **TAGE lookup.** For each of the 12 tables compute an index (PC, folded global history, path history) and a tag. The longest matching table is the provider and the next longest the alternate (the bimodal table if none).
3. **TAGE prediction.** Use the provider's counter sign, except when the provider is newly allocated (weak counter) and the use_alt_on_na counter for its table group and the alternate's confidence indicate that the alternate is more reliable.
4. **Loop predictor.** If a loop entry matches the PC with full confidence and the WITHLOOP counter is non-negative, its prediction replaces TAGE's.
5. **Statistical corrector.** Sum the centred values $(2c+1)$ of 6-bit counters from 3 bias tables, 4 global-history, 3 path-history and 4 local-history GEHL tables, the IMLI-SIC table (weighted $\times 2$), the IMHIST table and the 2 backward-history tables.
6. **Final choice.** If the corrector's sign agrees, keep the prediction. Otherwise the corrector wins, except when TAGE is highly confident and $|sum| < \theta/4$ (keep TAGE), or $\theta/4 \le |sum| < \theta/2$ (a chooser counter decides), or TAGE has medium confidence and $|sum| < \theta/4$ (a second chooser decides).

Training (called immediately with the true outcome).

1. **Corrector.** Train the chooser counters; if the corrector was wrong or $|sum| < \theta$, move every selected counter towards the outcome and adapt θ.
2. **Loop predictor.** Update the matching entry's iteration count, trip count and confidence; on a TAGE misprediction, allocate an entry with probability 1/4.
3. **TAGE.** Update the provider counter (and the alternate when a weak provider mispredicted) and the useful bits; on a TAGE misprediction allocate up to two entries in longer-history tables; if the final prediction was right, allocate only with probability 1/32. A TICK counter periodically halves the useful bits.
4. **Histories.** Shift one bit into the 640-bit global history and one PC bit into the 27-bit path history; update the 16-bit local history.
5. **Loop-structure state.** A not-taken branch is marked conditional; if it is known to be backward it ends the inner loop: IMLI is reset and a 0 enters the backward history. The outcome of every branch known to be conditional is appended to IMHIST[IMLI].

The predictor is deterministic (fixed-seed xorshift for the probabilistic decisions).

3.4 Implementation and integration

- `branch/tagescl.bpred` (C++11, one predictor object per CPU) implements all TAGE-family variants; compile-time switches build the baselines and ablations from the same code. Wrappers `tfl_tage_scl.bpred`, `tage_scl.bpred` and `tage.bpred` select configurations for the standard `./build_champsim.sh <name> no no no no lru 1` workflow.

- The simulator core was changed only to print exact counters (`BP_STATS`, `BP_TYPE_MISSES`); the timing model is unchanged.
- The experiment binaries are built out-of-tree by `experiments/scripts/build.sh` with exactly the configuration of `build_champsim.sh <predictor> no no no no lru 1`; all seven binaries are linked against the same simulator object files.

4 Experimental methodology

4.1 Simulator configuration

- ChampSim 2021, 1 core (6-wide fetch, 352-entry ROB), no prefetchers at any level, LRU last-level cache; misprediction penalty as in the shipped configuration. Only the branch predictor differs between the seven configurations.
- Every predictor and every trace: **50M warm-up + 200M measured instructions** (instructions 0–250M of each trace), the protocol documented for the earlier study (DPC-3 practice). Every trace holds 2,000M instructions, so no run reaches the end of its trace (checked for every run).
- Binaries: g++ 15.2.0, `-O3 -std=c++11`; predictor source `tagescl.bpred` SHA-256 `3be5e685...`, identical to the frozen version of the earlier study. A smoke test (1M + 10M instructions on 657.xz_s-2302B) reproduced the earlier study's recorded IPC, cycle and misprediction counts exactly for both bimodal and TFL-TAGE-SC-L, confirming that the new toolchain does not change results.
- The 350 simulations (7 predictors × 50 traces) ran 14 at a time under WSL2 on a 16-thread Intel i7-11800H between 2026-10-31 21:48 and 2026-11-01 10:26 (UTC); 350 completed successfully and 0 failed (Section 4.4).

4.2 Benchmark trace description

The evaluation uses 50 SimPoint traces of the SPEC CPU2017 speed suite [15] in ChampSim format (20 benchmarks; 26 SPECint and 24 SPECfp traces):

- **Original 10** (the traces of the earlier study): 602.gcc_s-1850B, 605.mcf_s-1152B, 607.cactuBSSN_s-2421B, 619.lbm_s-2676B, 620.omnetpp_s-141B, 623.xalancbmk_s-10B, 625.x264_s-12B, 631.deepsjeng_s-928B, 638.imagick_s-824B, 657.xz_s-2302B.
- **Additional 40**: three or four SimPoints each of perlbench, bwaves, wrf, leela, nab, exchange2, fotonik3d and roms; two of cam4; one of pop2; and further SimPoints of gcc (2), mcf (2), cactuBSSN, lbm, omnetpp, xalancbmk, x264, imagick and xz (2).

All files come from the Zenodo record "SPEC CPU 2017 ChampSim Traces" [2] (file names as in the DPC-3 distribution [1]; 16.06 GB compressed). Before the experiment every file was checked with `xz -t` (full decompression and CRC check), its SHA-256 and MD5 were recorded, and size and MD5 were compared with Zenodo's published values: all 50 match, and the 50 SHA-256 values are distinct (no duplicates). One local file (an interrupted download of 644.nab_s-5853B, a byte-exact prefix of the complete file) failed `xz -t` and was excluded in favour of the complete copy. Appendix B lists the manifest; Table 8 in Section 6.3 gives per-trace characteristics.

4.3 Metrics

- **IPC** = measured instructions / measured cycles (ChampSim's "CPU 0 cumulative IPC" of the region of interest).
- **MPKI** = 1000× mispredictions / measured instructions, all branch types (ChampSim's definition; recomputed from exact counters and checked against the printed value).

- **Accuracy** = (branches − mispredictions) / branches. ChampSim counts *all branch types*; because unconditional branches are trivially predicted, conditional-branch accuracy is reported separately. Aggregate accuracy is *pooled*: total correct predictions over total branches of all traces.
- **Mean IPC** is the arithmetic mean of the per-trace IPCs; **geometric-mean IPC** is the geometric mean of the per-trace IPCs; the **speedup** of predictor A over B is the geometric mean of the per-trace ratios $\mathrm{IPC}_A/\mathrm{IPC}_B$ (equal to the ratio of the geometric-mean IPCs); *IPC improvement* = (speedup − 1) × 100%. The arithmetic mean is dominated by high-IPC traces, the geometric mean weighs relative changes equally; both are reported, and conclusions are drawn from the geometric mean.
- **Mean MPKI** is the arithmetic mean over traces; percentage MPKI reductions are computed from the means, $(\mathrm{MPKI}_{\mathrm{ref}} - \mathrm{MPKI})/\mathrm{MPKI}_{\mathrm{ref}}$, not averaged per trace (near-zero-MPKI traces would otherwise dominate).
- **Per-trace wins/losses** count traces with strictly fewer (more) mispredictions or strictly fewer (more) cycles; exact equality is a tie.
- **Storage** is the logical predictor state in bits (Section 5).

4.4 Validity checks

All raw outputs were re-parsed by an independent script (`experiments/scripts/analyze_50trace.py`, standard library only) which records one row per expected run, including failures, and checks: exit status, completion, requested window, no trace wrap-around, IPC/MPKI/accuracy recomputed from the exact counters against ChampSim's printed values, the predictor-reported storage against Table 4, identical branch counts for all predictors on the same trace, the binary hash of every run against the binaries in `build/bin`, and that every summary value can be recomputed from `raw_results.csv`. The check log is `results_50trace/checks.txt`. All 25 checks passed (0 failed): every one of the 350 runs completed with exit status 0 over the full requested window, and all seven predictors ran on all 50 traces, so every aggregate in this report is over the same 50 traces. In addition, the 70 runs on the original ten traces are identical, in cycles and mispredictions, to the corresponding runs of the earlier study (Section 8).

5 Hardware and storage accounting

Table 4: Storage of every predictor (logical state). TAGE-family values are printed by the predictor and were checked in every run.

Predictor	Bits	KB	Basis
Bimodal	32,768	4.00	16,384 x 2-bit counters (source count)
Gshare	32,782	4.00	16,384 x 2-bit counters + 14-bit global history (source count)
Perceptron	32,624	3.98	163 perceptrons x 25 x 8-bit weights + 24-bit history (source count)
Hashed perceptron	524,542	64.03	16 x 4,096 x 8-bit weights + 240-bit history + 8-bit theta (assumed width) + 6-bit training counter (source count)
TAGE	383,674	46.84	storage_bits() printed by the predictor
TAGE-SC-L	476,319	58.14	storage_bits() printed by the predictor
TFL-TAGE-SC-L	508,666	62.09	storage_bits() printed by the predictor

Storage is the logical state a hardware implementation would hold, in bits. For the TAGE family the predictor counts its own state (`storage_bits()`) and prints it at start-up; the printed value

was checked in every one of the corresponding runs. The ChampSim predictors print nothing, so their state was counted from the source code: tables and history registers are counted at their declared widths, the 8-bit width of the hashed perceptron's adaptive threshold is an assumption, and the perceptron's speculative copy of the history is not counted because it is identical to the architectural copy under the immediate update of this ChampSim. Table 3 gives the breakdown of TFL-TAGE-SC-L.

Three storage classes result: about 4 KB (bimodal, gshare, perceptron), 47 KB (TAGE) and 58–64 KB (TAGE-SC-L, TFL-TAGE-SC-L, hashed perceptron). TFL-TAGE-SC-L uses 508,666 bits (62.09 KB): 58.14 KB for the TAGE-SC-L part plus 32,347 bits (3.95 KB, +6.8%) for the loop-structure components and the inference tables. It is 15,876 bits smaller than the 64 KB hashed perceptron (524,542 bits). The like-for-like comparisons are therefore with the hashed perceptron (largest ChampSim predictor, similar budget) and TAGE-SC-L (same implementation without the contribution); the 4 KB predictors are included because they are ChampSim's defaults, not as storage-equivalent competitors.

Storage in bits is not silicon area. ChampSim models neither predictor access latency nor area or energy, so these were not measured. Qualitatively, TFL-TAGE-SC-L reads 13 TAGE tables, a loop table and 18 corrector tables per prediction (the hashed perceptron reads 16 tables); TAGE-SC-L designs are normally pipelined over two cycles with an overriding scheme, and the extra corrector inputs lengthen the adder tree by about one level.

6 Results

6.1 Overall comparison

Table 5: Results over the 50 traces (50M + 200M instructions each). Bold: best value in the column.

Predictor	Storage (KB)	Mean IPC	Geomean IPC	Speedup vs Bimodal (%)	Mean MPKI	Accuracy all (%)	Accuracy cond. (%)
Bimodal	4.00	0.9065	0.7797	+0.00	9.334	93.089	91.476
Gshare	4.00	0.9825	0.8330	+6.84	4.619	96.580	95.964
Perceptron	3.98	1.0060	0.8508	+9.13	3.700	97.260	96.676
Hashed perceptron	64.03	1.0539	0.8845	+13.45	1.922	98.577	98.244
TAGE	46.84	1.0597	0.8877	+13.86	1.805	98.663	98.352
TAGE-SC-L	58.14	1.0636	0.8909	+14.27	1.663	98.769	98.481
TFL-TAGE-SC-L (proposed)	62.09	**1.0641**	**0.8914**	**+14.33**	**1.628**	**98.795**	98.512

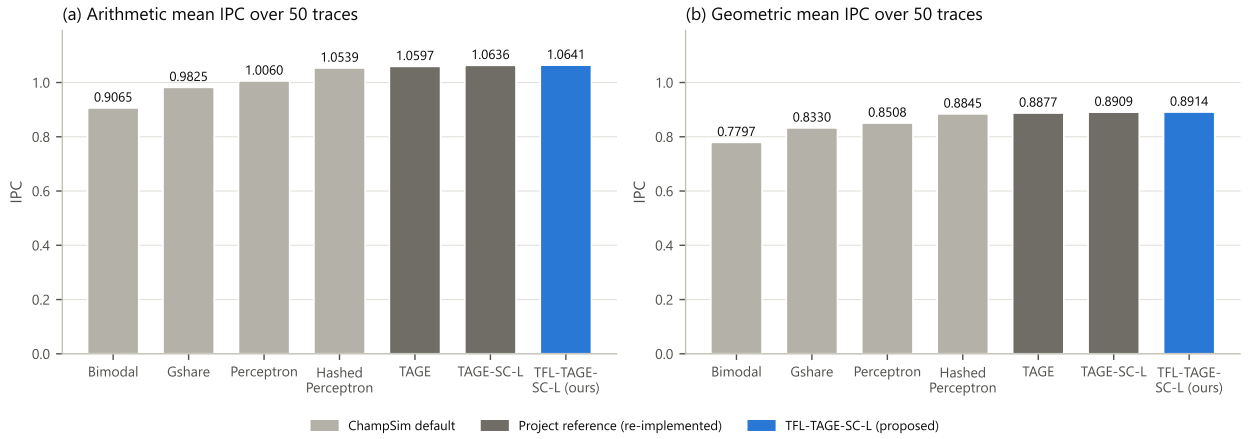


Figure 1: Arithmetic-mean (a) and geometric-mean (b) IPC of every predictor over 50 traces.

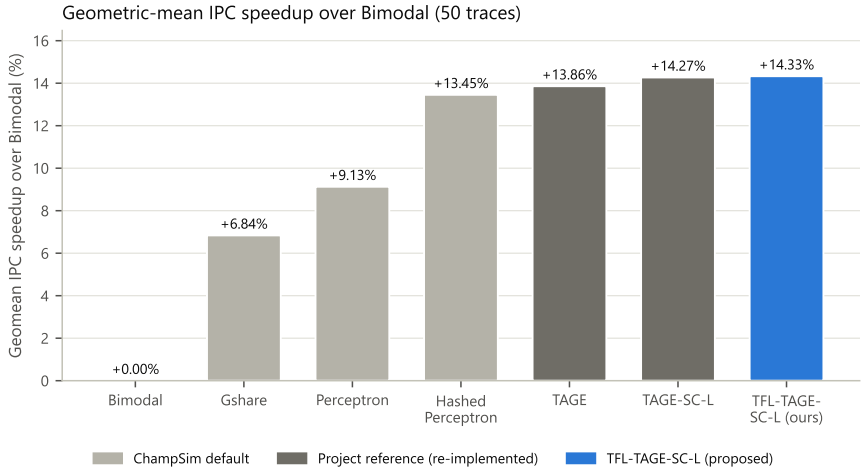


Figure 2: Geometric-mean IPC speedup over bimodal.

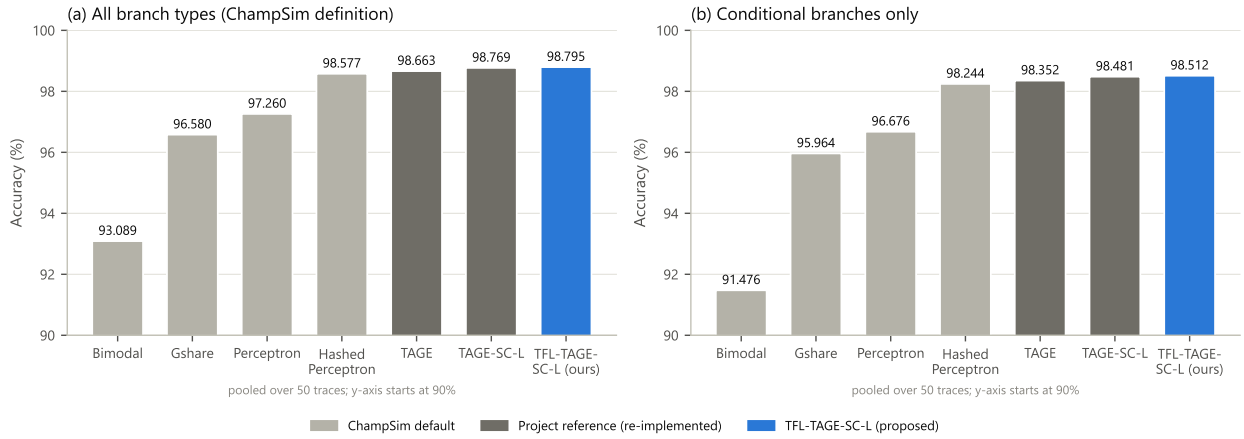


Figure 3: Pooled branch prediction accuracy: all branch types (ChampSim's definition) and conditional branches only. The y-axis does not start at zero.

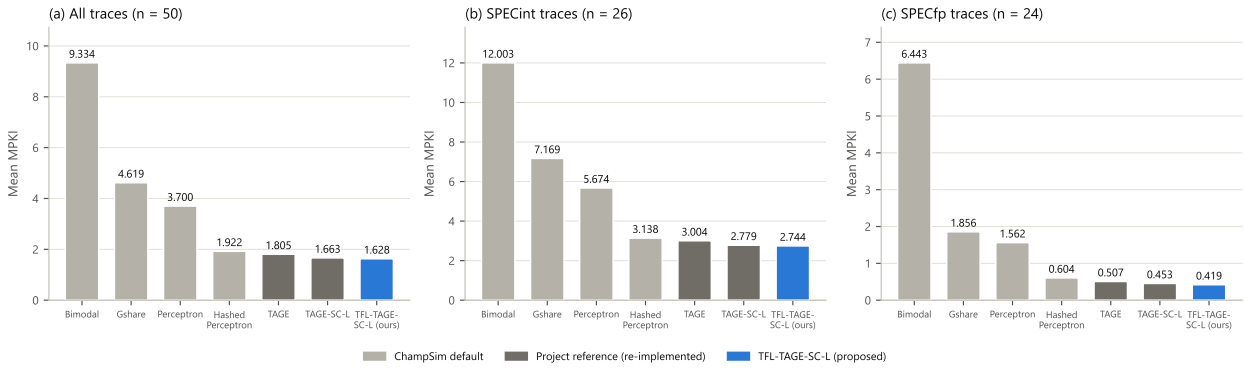


Figure 4: Mean MPKI over all traces, the SPECint traces and the SPECfp traces (note the different y-scales).

Table 5 and Figures 1–4 summarise the 50 traces. TFL-TAGE-SC-L ranks first of the seven predictors on every aggregate: geometric-mean IPC (0.8914), arithmetic-mean IPC (1.0641), pooled accuracy over all branches (98.795%) and over conditional branches (98.512%), and mean MPKI (1.628).

The predictors form three clear tiers. The 4 KB ChampSim predictors are far behind: bimodal (9.334 MPKI), gshare (4.619) and perceptron (3.700). The hashed perceptron (1.922) is the best ChampSim predictor and is close to the TAGE family: TAGE (1.805), TAGE-SC-L (1.663) and TFL-TAGE-SC-L (1.628). Relative to bimodal, the geometric-mean speedup grows from +6.84% (gshare) and +9.13% (perceptron) to +13.45% (hashed perceptron), +13.86% (TAGE), +14.27% (TAGE-SC-L) and +14.33% (TFL-TAGE-SC-L) (Figure 2). Most of the available IPC gain is therefore obtained by any of the large predictors. The differences among the top four are an order of magnitude smaller than the gap to the 4 KB predictors.

The arithmetic and geometric means give the same ranking. The arithmetic means are higher because the per-trace IPCs span more than an order of magnitude: from about 2 (648.exchange2) to well below 0.5 on the memory-bound traces (605.mcf, 620.omnetpp, 623.xalancbmk, one 602.gcc window), whose IPC is limited by cache misses whatever the predictor (Appendix A). Accuracy differences look small in percentage points because most branches are easy; MPKI expresses the same difference in the units that matter for performance. The hashed perceptron's 98.577% and TFL-TAGE-SC-L's 98.795% correspond to a 15.29% difference in mean MPKI.

The per-predictor means hide a very uneven distribution over traces (Figure 7). With TFL-TAGE-SC-L, 20 traces have an MPKI below 0.1, and 5 have an MPKI above 5. The five traces with the highest MPKI (605.mcf_s-1152B, 641.leela_s-149B, 641.leela_s-602B, 657.xz_s-2302B, 641.leela_s-334B) account for 60.3% of TFL-TAGE-SC-L's mean MPKI; all are SPECint traces (mcf, leela, xz). Mean MPKI is therefore dominated by a few hard traces, a fact that matters when interpreting the comparisons below.

6.2 TFL-TAGE-SC-L relative to each baseline

Table 6: TFL-TAGE-SC-L relative to each baseline over 50 traces (MPKI reductions from the means; IPC improvement from the geometric-mean speedup; per-trace counts by exact mispredictions and cycles).

Baseline	Baseline mean MPKI	MPKI reduction	MPKI reduction (%)	Geomean IPC improvement (%)	Traces: lower / equal / higher MPKI	Traces: higher / equal / lower IPC
Bimodal	9.334	7.7064	82.56	+14.329	49 / 1 / 0	47 / 1 / 2
Gshare	4.619	2.9908	64.76	+7.013	49 / 1 / 0	48 / 1 / 1
Perceptron	3.700	2.0727	56.01	+4.767	49 / 1 / 0	48 / 1 / 1
Hashed perceptron	1.922	0.2939	15.29	+0.772	46 / 2 / 2	45 / 2 / 3
TAGE	1.805	0.1773	9.82	+0.413	46 / 3 / 1	46 / 3 / 1
TAGE-SC-L	1.663	0.0347	2.09	+0.052	29 / 5 / 16	30 / 5 / 15

Table 6 compares TFL-TAGE-SC-L with each baseline.

ChampSim's 4 KB predictors. Against bimodal, gshare and perceptron, TFL-TAGE-SC-L reduces mean MPKI by 82.56%, 64.76% and 56.01%, and improves geometric-mean IPC by +14.329%, +7.013% and +4.767%. It has fewer mispredictions on 49 of 50 traces against each of them. The one remaining trace is a tie: on 649.fotonik3d_s-1176B every predictor makes exactly the same number of mispredictions. These comparisons are not storage-equivalent (about 4 KB against 62 KB) and are included because these are the default predictors of the simulator.

Hashed perceptron (best ChampSim predictor, 64 KB). This is the most relevant comparison with a ChampSim predictor, because its storage is similar and slightly larger. TFL-TAGE-SC-L reduces mean MPKI from 1.922 to 1.628 (15.29%) and improves geometric-mean IPC by +0.772% (arithmetic mean +0.973%). The gain is broad rather than driven by a few traces:

- TFL-TAGE-SC-L has fewer mispredictions on 46 traces and more on 2 (2 ties).
- It has higher IPC on 45 traces and lower IPC on 3. No IPC loss exceeds 0.1%: the largest is -0.02% on 619.lbm_s-2676B.
- 33 traces gain more than 0.1% IPC, and the median per-trace IPC change is +0.435%.

- The largest single contributor (641.leela_s-602B) provides only 7.2% of the MPKI reduction; without it the reduction is still 15.85%.

TAGE. Against our TAGE with the same tagged tables but no corrector or loop predictor, TFL-TAGE-SC-L reduces mean MPKI by 9.82% and improves geometric-mean IPC by +0.413%. It has fewer mispredictions on 46 traces and more on 1. This difference combines the statistical corrector, the loop predictor and the loop-structure components.

TAGE-SC-L (the same predictor without the contribution). This comparison isolates the effect of the proposed loop-structure components and the target-free inference. The effect is positive but small:

- Mean MPKI falls from 1.663 to 1.628 (2.09%). Pooled accuracy rises by +0.026 percentage points, and geometric-mean IPC by +0.052%.
- TFL-TAGE-SC-L has fewer mispredictions on 29 traces, more on 16 and the same number on 5.
- For IPC the count is 30 higher, 15 lower and 5 equal.

The losses are all tiny: the largest MPKI increase on any trace is 0.0026 (644.nab_s-9322B), and the largest IPC loss is -0.01% (621.wrf_s-6673B). The gains, in contrast, are concentrated:

- On 627.cam4_s-490B MPKI falls by 0.750 and IPC rises by +1.17%. This one trace provides 43.2% of the mean-MPKI reduction; without it, the reduction over the other 49 traces is 1.26%.
- On 28 traces the IPC changes by less than 0.01% in either direction, and the median per-trace IPC change is +0.005%.

The improvement over TAGE-SC-L is therefore consistent in sign but modest in size on most workloads, and larger on a few.

6.3 Per-trace analysis

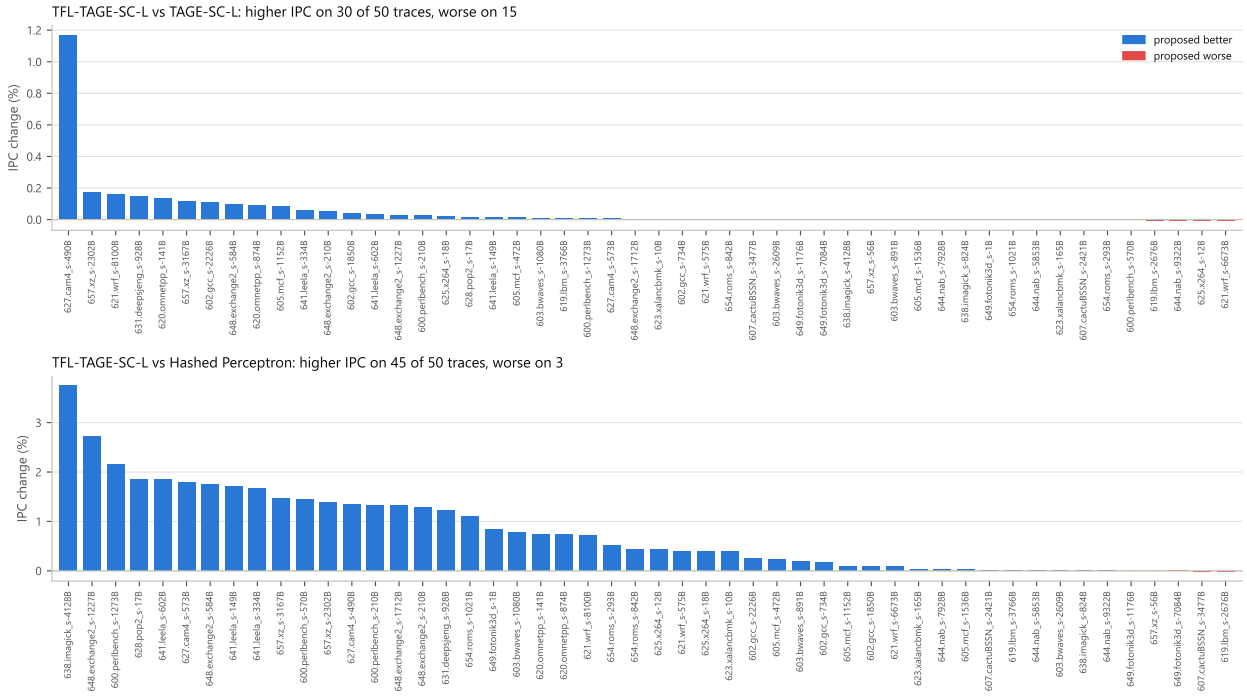


Figure 5: Per-trace IPC change of TFL-TAGE-SC-L relative to the best baseline by geometric-mean IPC and the best ChampSim default predictor, sorted.

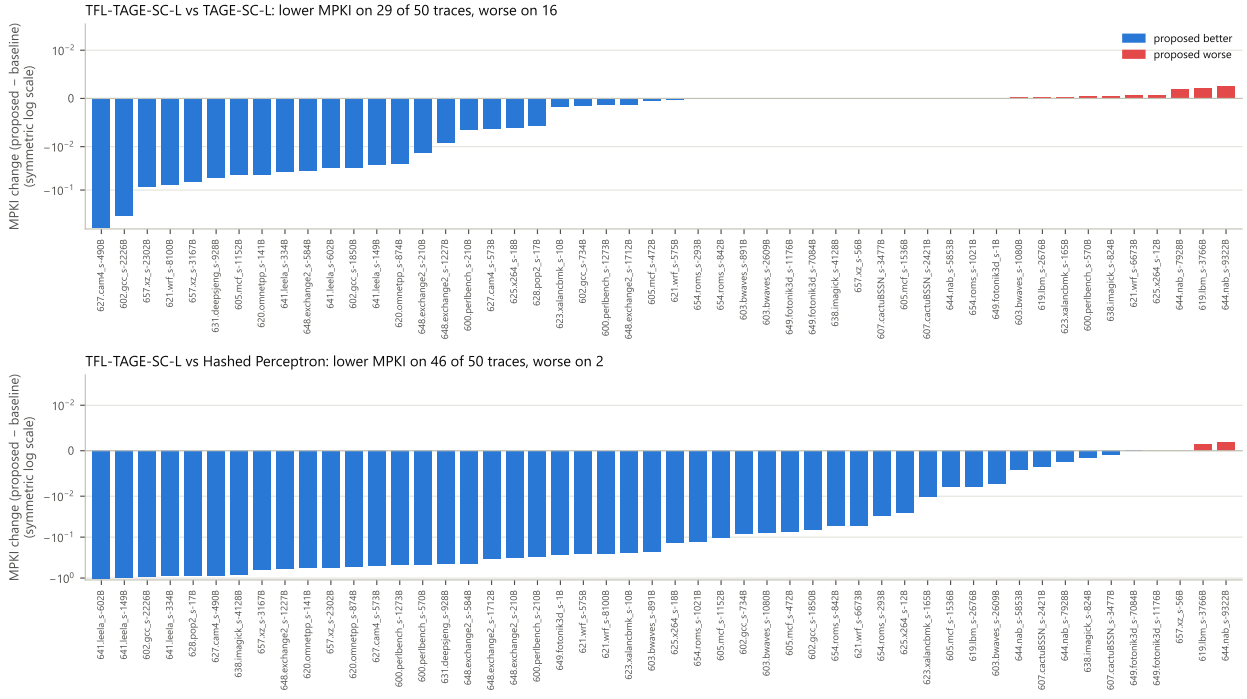


Figure 6: Per-trace MPKI change of TFL-TAGE-SC-L (negative = fewer mispredictions; symmetric-log scale, linear within ±0.01).

Table 7: Traces with the largest IPC gains and losses of TFL-TAGE-SC-L relative to the best baseline and the best ChampSim default (ΔMPKI = proposed − baseline).

	Largest IPC gains			Lowest IPC changes (losses, if any)		
Baseline	Trace	ΔIPC (%)	ΔMPKI	Trace	ΔIPC (%)	ΔMPKI
TAGE-SC-L	627.cam4_s-490B	+1.171	-0.7499	621.wrf_s-6673B	-0.009	+0.0007
	657.xz_s-2302B	+0.176	-0.0869	625.x264_s-12B	-0.009	+0.0007
	621.wrf_s-8100B	+0.160	-0.0750	644.nab_s-9322B	-0.008	+0.0026
	631.deepsjeng_s-928B	+0.148	-0.0520	619.lbm_s-2676B	-0.006	+0.0003
	620.omnetpp_s-141B	+0.138	-0.0438	600.perlbench_s-570B	-0.002	+0.0004
	657.xz_s-3167B	+0.115	-0.0656	654.roms_s-293B	-0.001	-0.0000
Hashed perceptron	638.imagick_s-4128B	+3.750	-0.8418	619.lbm_s-2676B	-0.018	-0.0080
	648.exchange2_s-1227B	+2.729	-0.5905	607.cactuBSSN_s-3477B	-0.009	-0.0009
	600.perlbench_s-1273B	+2.158	-0.4864	649.fotonik3d_s-7084B	-0.000	-0.0000
	628.pop2_s-17B	+1.845	-0.8691	657.xz_s-56B	+0.000	+0.0000
	641.leela_s-602B	+1.845	-1.0569	649.fotonik3d_s-1176B	+0.000	+0.0000
	627.cam4_s-573B	+1.791	-0.5023	644.nab_s-9322B	+0.000	+0.0019

Figures 5 and 6 show the per-trace changes relative to TAGE-SC-L (the best baseline by geometric-mean IPC) and the hashed perceptron (the best ChampSim predictor). Table 7 lists the extremes, and Table 8 lists every trace.

Best predictor per trace. TFL-TAGE-SC-L has the fewest mispredictions on 34 of the 50 traces: alone on 29, and tied with others on the rest. TAGE-SC-L is best or tied on 21 (alone on 16), and no other predictor is the sole best on any trace. On 5 traces several predictors make exactly the same, minimal number of mispredictions. On 649.fotonik3d_s-1176B all seven do; on 657.xz_s-56B the four large predictors make none at all.

By cycles, TFL-TAGE-SC-L is fastest or tied on 33 traces and TAGE-SC-L on 18. The hashed perceptron is fastest alone on 2 traces and TAGE on 1. These are traces with almost no mispredictions, where cycle counts differ by a few hundredths of a percent for reasons unrelated to prediction quality. For example, on 649.fotonik3d_s-7084B the hashed perceptron is fastest although it mispredicts more than the TAGE family.

Where the gains come from. Against the hashed perceptron, the largest IPC gains are on 638.imagick_s-4128B (+3.75%), 648.exchange2, 600.perlbench, 628.pop2 and 641.leela (Table 7). On these traces every TAGE-family predictor is clearly more accurate than the hashed perceptron, which suggests branch patterns that tagged long-history tables capture better than a hashed perceptron. On 638.imagick_s-4128B, for instance, TAGE-SC-L and TFL-TAGE-SC-L make almost no mispredictions, while the hashed perceptron does not reach that level (Figure 7).

Our TAGE alone has more mispredictions than the hashed perceptron on 9 traces. With the statistical corrector and loop predictor (TAGE-SC-L) that number falls to 0. For TFL-TAGE-SC-L it is 2, both by at most 0.0019 MPKI. The corrector is thus what makes the TAGE family robust on traces where a perceptron-style predictor does well.

Effect of the loop-structure components. Relative to TAGE-SC-L, the largest reductions are on:

- 627.cam4_s-490B (MPKI 0.750 lower);
- 602.gcc_s-2226B, both 657.xz traces, 621.wrf_s-8100B, 631.deepsjeng, 605.mcf_s-1152B and 620.omnetpp_s-141B.

In relative terms the largest reductions are on 602.gcc. On 602.gcc_s-2226B MPKI falls from 0.534 to 0.149 (72.1%), and on 602.gcc_s-1850B from 0.063 to 0.032 (49.8%). However, the median relative change over the 34 traces on which TAGE-SC-L has an MPKI of at least 0.05 is only 0.65%.

The 16 traces with more mispredictions than TAGE-SC-L belong to twelve benchmarks (the largest increases: 644.nab, 619.lbm, 625.x264, 621.wrf). Every increase is at most 0.0026 MPKI: at

most a few hundred extra mispredictions in 200M instructions, and fewer than 100 on most of these traces. These look like the expected side effect of adding inputs to the corrector (more aliasing and slower training), not a systematic failure on a workload class.

Hard traces. All predictors struggle on the same traces: 605.mcf_s-1152B, the three 641.leela traces, 657.xz_s-2302B, 631.deepsjeng and 620.omnetpp (Figure 7). There even TFL-TAGE-SC-L mispredicts several times per thousand instructions, and it removes at most about 1.5% of TAGE-SC-L's mispredictions. These programs (game-tree search, network simulation, graph optimisation, compression) are known for branches that depend on data values rather than on branch history. None of the evaluated predictors, including the proposed one, addresses that kind of branch.

Table 8: Per-trace characteristics and results (measured window). Br/KI = branches per 1,000 instructions; ΔIPC relative to TAGE-SC-L and the hashed perceptron.

Trace	Set	Br/KI	Cond. share	MPKI HP	MPKI TSL	MPKI proposed	ΔIPC (%) vs TSL	ΔIPC (%) vs HP
600.perlbench_s-1273B	add/INT	140.1	70%	1.120	0.635	0.634	+0.008	+2.158
600.perlbench_s-210B	add/INT	140.1	71%	0.954	0.666	0.659	+0.025	+1.323
600.perlbench_s-570B	add/INT	164.9	72%	1.534	1.051	1.051	-0.002	+1.454
602.gcc_s-1850B	orig/INT	256.4	83%	0.097	0.063	0.032	+0.039	+0.084
602.gcc_s-2226B	add/INT	356.4	98%	1.057	0.534	0.149	+0.114	+0.263
602.gcc_s-734B	add/INT	240.2	80%	0.235	0.155	0.153	+0.003	+0.179
603.bwaves_s-1080B	add/FP	145.3	90%	0.094	0.014	0.015	+0.010	+0.771
603.bwaves_s-2609B	add/FP	25.0	100%	0.007	0.000	0.000	+0.000	+0.007
603.bwaves_s-891B	add/FP	167.6	96%	0.223	0.000	0.000	-0.000	+0.190
605.mcf_s-1152B	orig/INT	258.4	74%	13.981	13.920	13.874	+0.087	+0.089
605.mcf_s-1536B	add/INT	155.8	57%	0.019	0.011	0.011	-0.000	+0.020
605.mcf_s-472B	add/INT	128.9	92%	1.106	1.033	1.033	+0.013	+0.240
607.cactuBSSN_s-2421B	orig/FP	1.4	52%	0.007	0.003	0.003	-0.001	+0.016
607.cactuBSSN_s-3477B	add/FP	8.9	61%	0.001	0.000	0.000	+0.000	-0.009
619.lbm_s-2676B	orig/FP	12.1	95%	0.109	0.101	0.101	-0.006	-0.018
619.lbm_s-3766B	add/FP	15.6	87%	0.366	0.365	0.367	+0.010	+0.013
620.omnetpp_s-141B	orig/INT	209.9	64%	3.838	3.322	3.278	+0.138	+0.749
620.omnetpp_s-874B	add/INT	208.0	64%	3.727	3.237	3.211	+0.095	+0.744
621.wrf_s-575B	add/FP	134.0	77%	0.475	0.224	0.223	+0.002	+0.398
621.wrf_s-6673B	add/FP	60.1	93%	0.079	0.026	0.027	-0.009	+0.083
621.wrf_s-8100B	add/FP	123.4	81%	1.166	0.993	0.918	+0.160	+0.713
623.xalancbmk_s-10B	orig/INT	258.7	90%	0.692	0.455	0.453	+0.005	+0.387
623.xalancbmk_s-165B	add/INT	277.6	95%	0.031	0.020	0.020	-0.001	+0.032
625.x264_s-12B	orig/INT	42.4	84%	0.079	0.053	0.054	-0.009	+0.431
625.x264_s-18B	add/INT	47.0	84%	0.662	0.533	0.527	+0.023	+0.391
627.cam4_s-490B	add/FP	96.0	95%	4.696	4.578	3.828	+1.171	+1.351
627.cam4_s-573B	add/FP	133.5	86%	0.980	0.484	0.478	+0.007	+1.791
628.pop2_s-17B	add/FP	152.5	93%	1.197	0.333	0.328	+0.018	+1.845
631.deepsjeng_s-928B	orig/INT	147.1	66%	4.166	3.761	3.709	+0.148	+1.221
638.imagick_s-4128B	add/FP	193.5	98%	0.842	0.000	0.000	+0.000	+3.750
638.imagick_s-824B	orig/FP	161.3	64%	0.090	0.088	0.089	-0.000	+0.003
641.leela_s-149B	add/INT	165.5	62%	10.929	9.962	9.936	+0.016	+1.703
641.leela_s-334B	add/INT	173.8	63%	8.304	7.463	7.425	+0.062	+1.670
641.leela_s-602B	add/INT	170.2	63%	10.043	9.018	8.986	+0.034	+1.845
644.nab_s-5853B	add/FP	96.8	69%	0.015	0.011	0.011	-0.001	+0.007
644.nab_s-7928B	add/FP	96.0	83%	1.488	1.484	1.486	-0.000	+0.021
644.nab_s-9322B	add/FP	128.8	81%	2.118	2.117	2.120	-0.008	+0.000
648.exchange2_s-1227B	add/INT	142.8	91%	1.372	0.791	0.782	+0.029	+2.729
648.exchange2_s-1712B	add/INT	134.1	92%	0.721	0.391	0.389	+0.005	+1.321
648.exchange2_s-210B	add/INT	136.2	92%	0.822	0.520	0.506	+0.057	+1.289
648.exchange2_s-584B	add/INT	140.7	90%	1.228	0.823	0.787	+0.100	+1.741
649.fotonik3d_s-1176B	add/FP	17.6	100%	0.004	0.004	0.004	+0.000	+0.000

Table 8: Per-trace characteristics and results (measured window). Br/KI = branches per 1,000 instructions; ΔIPC relative to TAGE-SC-L and the hashed perceptron.

Trace	Set	Br/KI	Cond. share	MPKI HP	MPKI TSL	MPKI proposed	ΔIPC (%) vs TSL	ΔIPC (%) vs HP
649.fotonik3d_s-1B	add/FP	237.8	66%	0.277	0.001	0.001	-0.000	+0.840
649.fotonik3d_s-7084B	add/FP	6.9	100%	0.000	0.000	0.000	+0.000	-0.000
654.roms_s-1021B	add/FP	150.1	83%	0.130	0.001	0.001	-0.001	+1.110
654.roms_s-293B	add/FP	35.5	100%	0.032	0.001	0.001	-0.001	+0.525
654.roms_s-842B	add/FP	164.4	81%	0.105	0.052	0.052	+0.001	+0.438
657.xz_s-2302B	orig/INT	125.0	85%	9.417	8.950	8.863	+0.176	+1.381
657.xz_s-3167B	add/INT	148.4	93%	5.445	4.878	4.813	+0.115	+1.465
657.xz_s-56B	add/INT	20.1	97%	0.000	0.000	0.000	+0.000	+0.000

6.4 Generalisation: original 10 versus additional 40 traces

Table 9: Mean MPKI and the gain of TFL-TAGE-SC-L on trace subsets (HP = hashed perceptron, TSL = TAGE-SC-L; IPC as geometric-mean improvement).

Subset	n	Mean MPKI			Proposed vs. baseline		
		Hashed perc.	TAGE-SC-L	Proposed	MPKI red. vs HP	MPKI red. vs TSL	IPC vs TSL
All traces	50	1.922	1.663	1.628	15.29%	2.09%	+0.052%
Original 10	10	3.247	3.072	3.046	6.22%	0.85%	+0.058%
Additional 40	40	1.590	1.310	1.273	19.93%	2.82%	+0.051%
SPECint	26	3.138	2.779	2.744	12.56%	1.26%	+0.049%
SPECfp	24	0.604	0.453	0.419	30.68%	7.61%	+0.056%

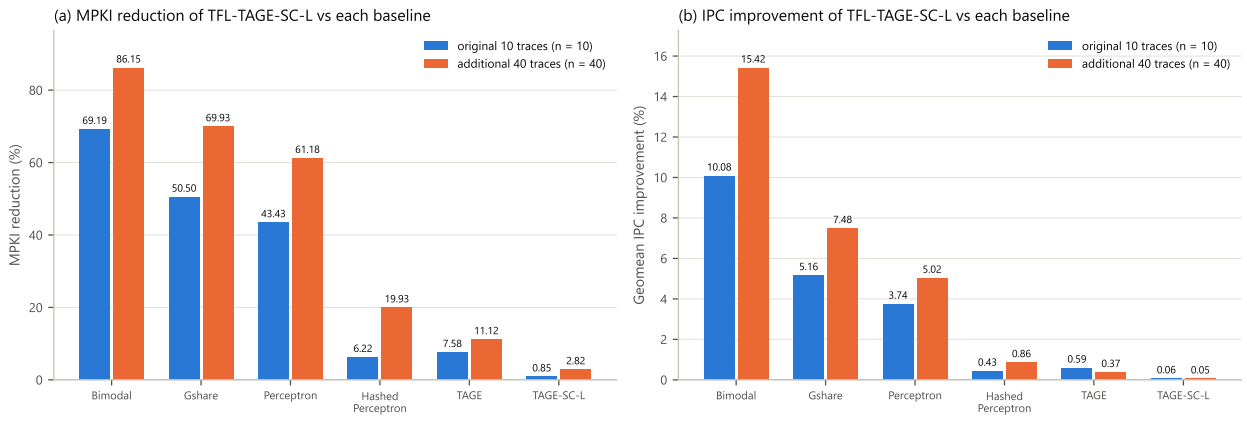


Figure 8: MPKI reduction and geometric-mean IPC improvement of TFL-TAGE-SC-L relative to each baseline, on the original 10 traces and on the 40 additional traces.

The design was developed on the original ten programs. Tuning used a different instruction window, but the final configuration was also checked on the reported window. The 40 additional traces were never used for design or tuning, which makes them the cleaner test (Table 9, Figure 8).

The result carries over. On the original ten traces TFL-TAGE-SC-L reduces mean MPKI by 6.22% relative to the hashed perceptron and by 0.85% relative to TAGE-SC-L, with geometric-mean IPC gains of +0.433% and +0.058%. These are exactly the values of the earlier study. On the 40 additional traces the corresponding values are:

- relative to the hashed perceptron: 19.93% lower MPKI and +0.856% higher geometric-mean IPC;

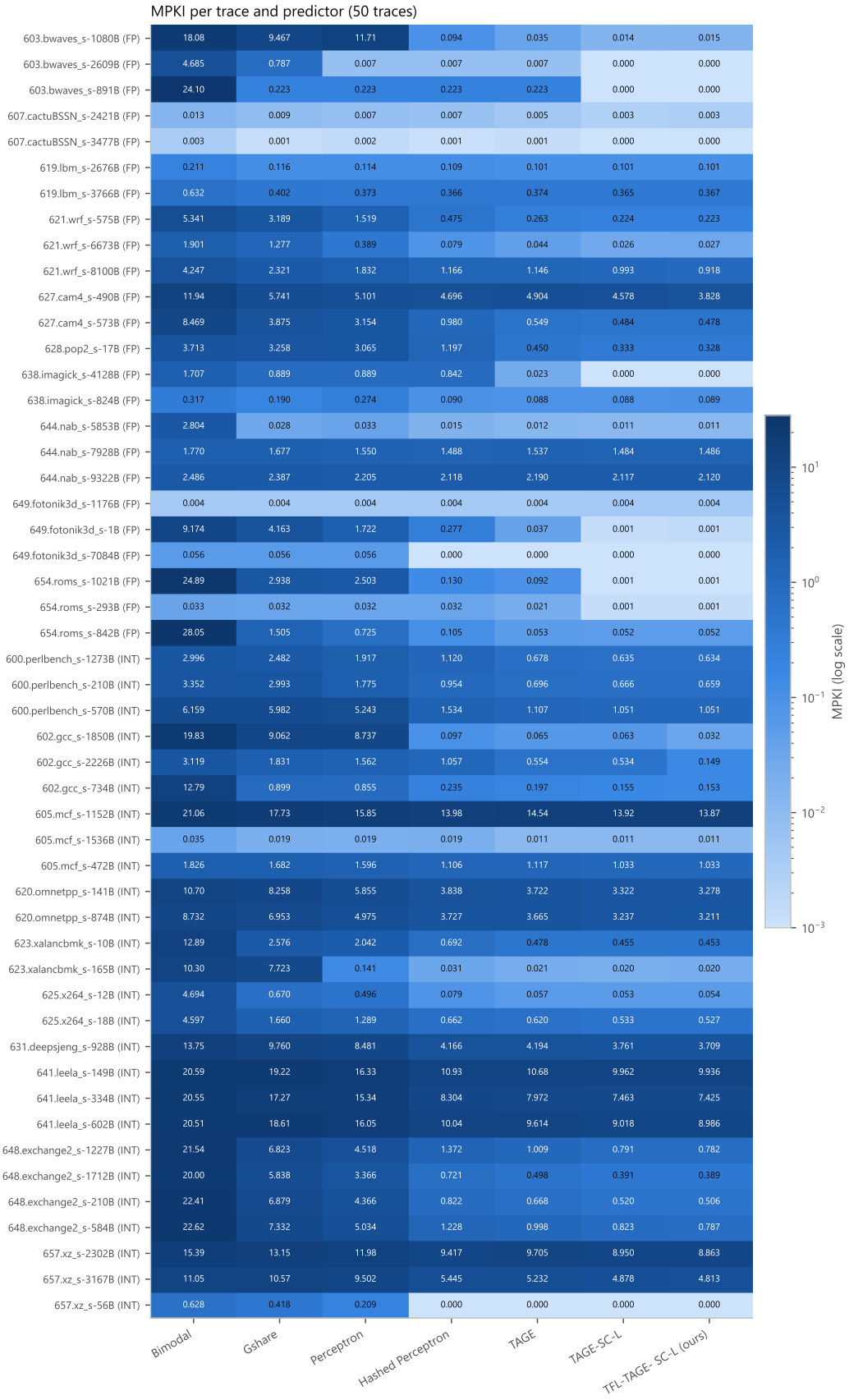


Figure 7: MPKI of every predictor on every trace (log colour scale), grouped into SPECfp and SPECint.

- relative to TAGE-SC-L: 2.82% lower MPKI and +0.051% higher geometric-mean IPC.

TFL-TAGE-SC-L has the lowest mean MPKI and the highest geometric-mean IPC in both subsets, so its first place does not depend on the traces used during design. The order among the baselines does change: TAGE is behind the hashed perceptron on the original ten but ahead on the additional traces (below).

Why the relative MPKI gain is larger on the additional traces. Two effects explain why the percentage reductions are larger on the new traces, while the IPC improvement over TAGE-SC-L is about the same in both subsets.

- The additional set contains many SPECfp traces with low MPKI (mean MPKI of the hashed perceptron 1.590 against 3.247 on the original ten). A small absolute reduction is therefore a larger percentage.
- It contains 627.cam4_s-490B, the trace with the largest gain over TAGE-SC-L.

The original ten traces are dominated by hard SPECint traces (605.mcf, 657.xz, 631.deepsjeng, 620.omnetpp). There, even our TAGE is slightly worse than the hashed perceptron (3.295 against 3.247 MPKI); on the additional traces TAGE is clearly better (1.433 against 1.590). The earlier study's conclusion that a TAGE of comparable size does not beat the hashed perceptron on average was thus a property of its ten traces and does not hold on the larger set.

SPECint and SPECfp. TFL-TAGE-SC-L reduces mean MPKI relative to TAGE-SC-L by 1.26% on the 26 SPECint traces and by 7.61% on the 24 SPECfp traces. Relative to the hashed perceptron the reductions are 12.56% and 30.68%. The geometric-mean IPC gain over TAGE-SC-L is similar in both suites (+0.049% and +0.056%). The gain over the hashed perceptron is larger on SPECint (+0.955% against +0.573%), where the absolute number of mispredictions removed is larger.

The overlap between the two subsets must be kept in mind: 12 of the 40 additional traces are further SimPoints of benchmarks that also appear among the original ten (for example 602.gcc, 605.mcf, 657.xz). They are different program phases, but not different programs. The other 28 come from ten benchmarks that the earlier study did not use at all.

6.5 Storage versus performance

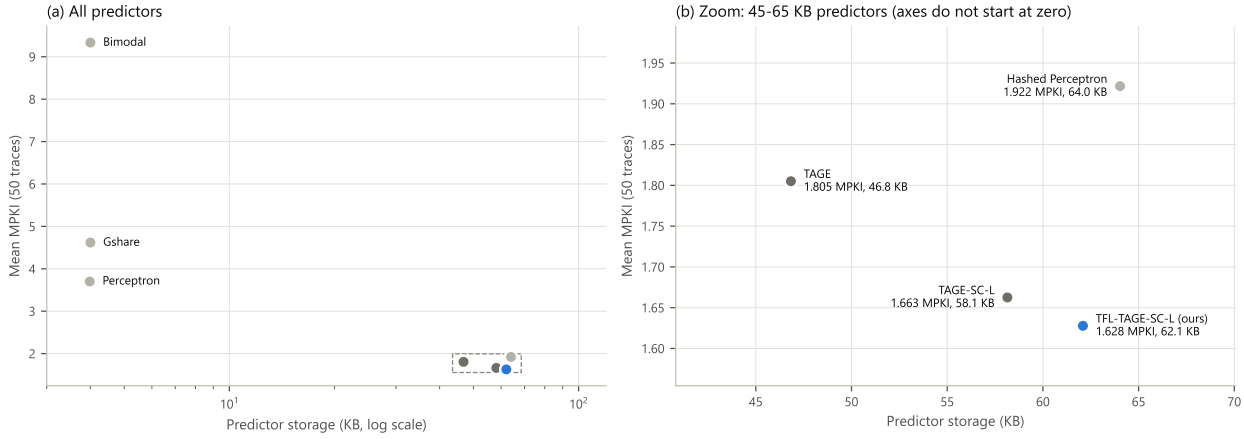


Figure 9: Storage against mean MPKI; (b) zooms on the 45–65 KB predictors.

Figure 9 places every predictor by storage and mean MPKI.

The three storage classes.

- **About 4 KB.** The perceptron is the most accurate (3.700 MPKI at 3.98 KB). It dominates gshare and bimodal, which use the same or slightly more storage.
- **47–64 KB.** The hashed perceptron is dominated by both TAGE (smaller and more accurate) and TFL-TAGE-SC-L (15,876 bits smaller, 15.29% lower MPKI).

Among the predictors evaluated, the best accuracy for a given storage is therefore obtained by the perceptron at about 4 KB, and above that by TAGE (46.84 KB), TAGE-SC-L (58.14 KB) and TFL-TAGE-SC-L (62.09 KB).

Diminishing returns along the TAGE family. Going from TAGE to TAGE-SC-L adds 58.14 − 46.84 KB (corrector and loop predictor) and lowers mean MPKI from 1.805 to 1.663. Going from TAGE-SC-L to TFL-TAGE-SC-L adds 3.95 KB (6.8%) and lowers it to 1.628 (2.09%). Per kilobyte, the loop-structure components are thus less effective than the corrector, which is consistent with their effect being concentrated on a few traces.

The equal-storage question. The experiment does not show whether spending the same 3.95 KB on larger TAGE-SC-L tables would give a similar gain: no storage-matched TAGE-SC-L was evaluated, neither here nor in the earlier study. Given the diminishing returns above and the small size of the gain, this is the main open question about the contribution. The comparison with the hashed perceptron, which is larger than TFL-TAGE-SC-L, is not affected by this caveat.

6.6 MPKI reduction versus IPC gain

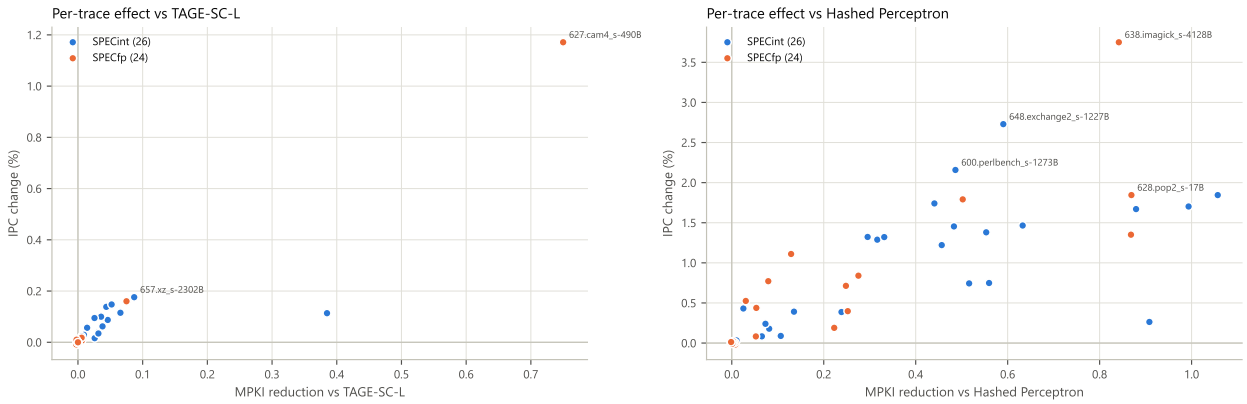


Figure 10: Per-trace MPKI reduction against IPC change of TFL-TAGE-SC-L, coloured by suite; the traces with the largest IPC change are labelled (labels that would overlap are omitted).

Figure 10 relates the per-trace MPKI reduction to the per-trace IPC change. Fewer mispredictions never cost measurable performance: no trace loses more than 0.1% IPC against any baseline. How much IPC an avoided misprediction buys, however, varies widely between traces.

- **Relative to the hashed perceptron**, IPC gain grows roughly with MPKI reduction, but with large scatter. On 638.imagick_s-4128B a reduction of 0.84 MPKI gives +3.75% IPC. Several SPECint traces with a similar or larger reduction gain about half as much.
- **Relative to TAGE-SC-L**, 627.cam4_s-490B stands out (0.750 MPKI, +1.17% IPC). On 602.gcc_s-2226B, by contrast, MPKI falls by 72.1% but IPC rises by only +0.114%.

The reason is the cost of a misprediction in this simulator. Fetch stops until the mispredicted branch executes, so the penalty depends on how long the branch waits for its operands, and on whether the pipeline is stalled on memory anyway. On 602.gcc_s-2226B the IPC with TFL-TAGE-SC-L is only 0.108: the core spends most of its time waiting for cache misses, and removing mispredictions recovers little. On compute-bound traces with high IPC, a misprediction wastes a full window of useful work, and the same MPKI reduction is worth more.

For this reason the report gives both MPKI (a property of the predictor) and IPC (its effect on this particular core), and uses the geometric mean for IPC.

6.7 Simulation runtime

Table 10: Simulation wall time per run (14 runs in parallel on 8 physical cores, so per-run times include contention).

Predictor	Runs ok	Mean wall time (min)	Max wall time (min)	Total (CPU-h)
Bimodal	50/50	28.8	310.8	24.0
Gshare	50/50	29.5	312.0	24.6
Perceptron	50/50	29.8	317.2	24.8
Hashed perceptron	50/50	30.6	317.6	25.5
TAGE	50/50	30.8	317.3	25.6
TAGE-SC-L	50/50	30.8	317.5	25.7
TFL-TAGE-SC-L	50/50	30.9	317.5	25.7

Total simulation effort was 176.0 CPU-hours of wall-clock time per job summed over the 350 successful runs. Runtime reflects simulation cost, not predictor latency: ChampSim does not model predictor access time.

7 Discussion

Which predictor is best? On these 50 traces TFL-TAGE-SC-L is the best of the seven predictors on every aggregate metric: geometric-mean IPC, arithmetic-mean IPC, pooled accuracy and mean MPKI. It is the sole best or tied on 34 traces by mispredictions. The margin over the strongest predictor shipped with ChampSim is clear and broad. Relative to the hashed perceptron:

- mean MPKI is 15.29% lower, and geometric-mean IPC +0.772% higher;
- TFL-TAGE-SC-L has fewer mispredictions on 46 of 50 traces, and no trace loses more than -0.02% IPC;
- it uses slightly less storage.

This is the main result of the evaluation, and it is a property of the TAGE-SC-L design as a whole. Our TAGE-SC-L without the contribution already has no more mispredictions than the hashed perceptron on any trace.

What does the contribution add? The loop-structure components made possible by the target-free inference reduce mean MPKI by 2.09% relative to the same predictor without them. They improve geometric-mean IPC by +0.052%. This is a small effect, and three facts qualify it:

- It is unevenly distributed: one trace (627.cam4_s-490B) provides 43.2% of the MPKI reduction, and the median per-trace IPC change is +0.005%.
- It is consistent in sign but not universal: fewer mispredictions on 29 traces and more on 16, every increase being at most 0.0026 MPKI.
- It costs 6.8% more storage, and a storage-matched TAGE-SC-L was not evaluated.

The aggregate IPC gain of +0.052% is more than ten times the largest run-to-run variation measured in the earlier study (at most 0.003% of geometric-mean IPC when the predictor's random seed is changed). That variation was measured on the original ten traces only, and with a single deterministic run per trace no statistical significance is claimed. The fair summary is that the components work as intended and help clearly on a few programs (627.cam4, 602.gcc, 657.xz). On most other programs they change little, so they are a refinement of TAGE-SC-L rather than a step beyond it. Which loop structures in these programs are responsible was not analysed.

Does the result generalise? Yes, in direction and ranking. On the 40 traces never used during design, TFL-TAGE-SC-L again has the lowest MPKI and the highest geometric-mean IPC, and its relative gains are larger than on the original ten. The IPC gain over TAGE-SC-L is practically the same in both subsets (+0.058% and +0.051%). The larger set also corrects a conclusion of the earlier study, which was based on the original ten traces. On those traces our TAGE was slightly worse than the hashed perceptron. Over all 50 traces TAGE is better (1.805 against 1.922 MPKI), and is worse on only 9 traces.

Arithmetic versus geometric means. Both means give the same ranking of all seven predictors. The relative differences between the top predictors are also similar under both: +0.049% (arithmetic) against +0.052% (geometric) relative to TAGE-SC-L, and +0.973% against +0.772% relative to the hashed perceptron. The conclusions therefore do not depend on the choice of mean.

Practical significance. For a designer choosing among the predictors shipped with ChampSim, the +0.772% IPC improvement over the hashed perceptron at slightly lower storage is the practically relevant result. The further +0.052% over TAGE-SC-L would justify its cost in a real design only if it were confirmed on a wider workload set. It would also need to be compared against the same storage spent on larger tables. In a processor with wrong-path fetch, the inference would further require checkpointing and repair of the loop-structure state.

8 Relation to the earlier 10-trace study

The earlier study evaluated the same predictor on the ten original traces with the same window and simulator configuration; its report, figures, workbook and audit are preserved unchanged in `archive/10trace_study/`.

The new experiment re-simulated the original ten traces from scratch rather than reusing the old outputs, with newly built binaries (a different compiler version) and the trace files from Zenodo. For these 70 predictor–trace combinations the new runs are *identical* to the earlier study in both cycle and misprediction counts (0 differ). The original-10 rows of Table 9 are therefore exactly the earlier study's values, now measured again, while every aggregate quoted elsewhere in this report is over all 50 traces. The earlier ablation study (functional replay harness) and its exploration results were not repeated; they are reported only in the archived report.

9 Limitations and threats to validity

- **Workloads.** 50 SimPoint windows of 20 SPEC CPU2017 speed benchmarks, 200M measured instructions each. Several benchmarks contribute three or four windows, so they carry more weight in the means than benchmarks with one window. Other suites (server, mobile, CBP traces) were not evaluated.
- **Statistics.** One deterministic simulation per predictor and trace. No claim of statistical significance is made; per-trace win/loss counts describe these runs, not a population. The earlier study measured run-to-run noise by changing the predictor's random seed: the standard deviation of mean MPKI was at most 0.03% of the mean (replay harness), and a different seed moved a design's geometric-mean IPC by at most 0.003% in ChampSim. Differences of that size should be read as noise.
- **Tuning.** The design was tuned on a different window (instructions 1000M–1150M) of the original ten programs, and its final configuration was also checked on the reported window of those ten traces. The 40 additional traces were never used for tuning; they are the cleaner test of generalisation (Section 6.4).
- **Simulator.** ChampSim 2021: direction prediction only, immediate predictor update, no wrong-path execution, no BTB or indirect-target modelling; all branch types count in accuracy

and MPKI. Absolute values are not comparable with newer ChampSim versions or with CBP results.

- **Inference.** The backward-branch inference is exact only because this ChampSim presents every branch in program order without wrong-path fetch; a speculative front end would need checkpointing and repair of the loop-structure state.
- **Re-implementation.** Our TAGE-SC-L (12 tagged tables, about 58 KB) is not Seznec's championship code; no claim is made relative to published state-of-the-art predictors evaluated on other simulators, traces or budgets.
- **Storage** is a logical bit count; the hashed perceptron's threshold width is an assumption. Area, energy and access latency were not modelled.

10 Conclusion and future work

TFL-TAGE-SC-L was evaluated, unchanged from its frozen design, on 50 SPEC CPU2017 traces against all four predictors shipped with ChampSim and against our own TAGE and TAGE-SC-L. All 350 simulations succeeded and passed the consistency checks.

- **Best overall.** TFL-TAGE-SC-L is the best of the seven predictors in geometric-mean IPC (0.8914), pooled accuracy (98.795%) and mean MPKI (1.628).
- **Against the best ChampSim predictor.** Relative to the 64 KB hashed perceptron, TFL-TAGE-SC-L lowers mean MPKI by 15.29% and raises geometric-mean IPC by +0.772%, with fewer mispredictions on 46 of 50 traces and slightly less storage. Against the 4 KB default predictors the gains are +4.767% to +14.329% in geometric-mean IPC.
- **The contribution itself.** Relative to the same TAGE-SC-L without them, the target-free loop-structure components lower mean MPKI by 2.09% and raise geometric-mean IPC by +0.052%, for 6.8% more storage. They have fewer mispredictions on 29 traces and more on 16, and 43.2% of the MPKI gain comes from one trace. The contribution is real on these runs but small.
- **Generalisation.** The first place of TFL-TAGE-SC-L and the direction of every one of its gains, observed on the original ten traces, hold on the 40 traces not used during design.

Future work. Four follow-ups address the open questions of this evaluation:

- a storage-matched TAGE-SC-L (the 3.95 KB spent on larger tables), to separate the value of the loop-structure history from the value of extra storage;
- an analysis of which loop structures produce the large gains on 627.cam4 and 602.gcc, and why some traces lose a few hundred predictions;
- evaluation on non-SPEC workloads (server, CBP and mobile traces) and in a ChampSim version with wrong-path execution, where the inference needs repair after mispredictions;
- modelling the access latency of the larger corrector, which this simulator does not capture.

References

[1] DPC-3 ChampSim traces of SPEC CPU2017. Third Data Prefetching Championship, 2019. `https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/`.

[2] SPEC CPU 2017 ChampSim traces. Zenodo, record 10960004, version v1, licence CC-BY-4.0, 2024. `https://zenodo.org/records/10960004`.

[3] Rahul Bera. ChampSim2, pull request #2: Merging many development features to mainline (CBP-6 TAGE-SC-L adapter). GitHub, 2026.

[4] Nathan Gober, Gino Chacon, Lei Wang, Paul V. Gratz, Daniel A. Jiménez, Elvira Teran, Seth Pugsley, and Jinchun Kim. The championship simulator: Architectural simulation for education and competition, 2022.

[5] Daniel A. Jiménez. Multiperspective perceptron predictor. In *5th JILP Workshop on Computer Architecture Competitions: Championship Branch Prediction (CBP-5)*, 2016.

[6] Daniel A. Jiménez and Calvin Lin. Dynamic branch prediction with perceptrons. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, pages 197–206, 2001.

[7] Chit-Kwan Lin and Stephen J. Tarsa. Branch prediction is not a solved problem: Measurements, opportunities, and future directions. In *IEEE International Symposium on Workload Characterization (IISWC)*, 2019. arXiv:1906.08170.

[8] Pierre Michaud. An alternative TAGE-like conditional branch predictor. *ACM Transactions on Architecture and Code Optimization*, 15(3), 2018.

[9] David Schall, Andreas Sandberg, and Boris Grot. The last-level branch predictor. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.

[10] André Seznec. A new case for the TAGE branch predictor. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 117–127, 2011.

[11] André Seznec. TAGE-SC-L branch predictors. In *4th JILP Workshop on Computer Architecture Competitions: Championship Branch Prediction (CBP-4)*, 2014.

[12] André Seznec. TAGE-SC-L branch predictors again. In *5th JILP Workshop on Computer Architecture Competitions: Championship Branch Prediction (CBP-5)*, 2016.

[13] André Seznec and Pierre Michaud. A case for (partially) TAgged GEometric history length branch prediction. *Journal of Instruction-Level Parallelism*, 8, 2006.

[14] André Seznec, Joshua San Miguel, and Jorge Albericio. The inner most loop iteration counter: A new dimension in branch history. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO)*, pages 347–357, 2015.

[15] Standard Performance Evaluation Corporation. SPEC CPU® 2017 benchmark suite. `https://www.spec.org/cpu2017/`, 2017.

A IPC, MPKI and accuracy of every predictor on every trace

Table 11: IPC of every predictor on every trace. Bim. = bimodal, Perc. = perceptron, Hashed P. = hashed perceptron, TSL = TAGE-SC-L, Proposed = TFL-TAGE-SC-L; bold = best on the trace.

Trace	Bim.	Gshare	Perc.	Hashed P.	TAGE	TSL	Proposed
600.perlbench_s-1273B	1.3808	1.4042	1.4339	1.4895	1.5186	1.5215	**1.5216**
600.perlbench_s-210B	1.3853	1.4092	1.4781	1.5412	1.5591	1.5612	**1.5616**
600.perlbench_s-570B	0.8886	0.8925	0.9088	0.9951	1.0081	**1.0096**	1.0095
602.gcc_s-1850B	0.2809	0.3178	0.3190	0.3470	0.3471	0.3471	**0.3473**
602.gcc_s-2226B	0.1073	0.1079	0.1080	0.1082	0.1083	0.1083	**0.1085**
602.gcc_s-734B	0.4370	0.5226	0.5229	0.5280	0.5286	0.5290	**0.5290**
603.bwaves_s-1080B	1.1996	1.3123	1.2971	1.5314	1.5409	1.5431	**1.5432**

Table 11: IPC of every predictor on every trace. Bim. = bimodal, Perc. = perceptron, Hashed P. = hashed perceptron, TSL = TAGE-SC-L, Proposed = TFL-TAGE-SC-L; bold = best on the trace.

Trace	Bim.	Gshare	Perc.	Hashed P.	TAGE	TSL	Proposed
603.bwaves_s-2609B	0.7288	0.7301	0.7351	0.7351	0.7351	**0.7351**	0.7351
603.bwaves_s-891B	0.4920	0.5391	0.5391	0.5391	0.5391	**0.5401**	0.5401
605.mcf_s-1152B	0.2908	0.2883	0.3032	0.3122	0.3073	0.3122	**0.3124**
605.mcf_s-1536B	0.1504	0.1505	0.1505	0.1505	0.1505	**0.1505**	0.1505
605.mcf_s-472B	0.2762	0.2759	0.2769	0.2916	0.2904	0.2923	**0.2923**
607.cactuBSSN_s-2421B	1.1950	1.1952	1.1952	1.1953	1.1954	**1.1955**	1.1955
607.cactuBSSN_s-3477B	1.1688	1.1688	1.1688	**1.1689**	1.1688	1.1688	1.1688
619.lbm_s-2676B	0.6331	0.6331	0.6331	0.6331	**0.6332**	0.6331	0.6330
619.lbm_s-3766B	0.4689	0.4689	0.4689	0.4689	0.4689	0.4689	**0.4689**
620.omnetpp_s-141B	0.3119	0.3234	0.3310	0.3388	0.3391	0.3408	**0.3413**
620.omnetpp_s-874B	0.3361	0.3468	0.3542	0.3595	0.3597	0.3619	**0.3622**
621.wrf_s-575B	1.1071	1.1297	1.1615	1.1839	1.1879	1.1886	**1.1886**
621.wrf_s-6673B	0.7895	0.7914	0.7948	0.7968	0.7974	**0.7976**	0.7975
621.wrf_s-8100B	1.1458	1.1788	1.1897	1.2101	1.2113	1.2168	**1.2187**
623.xalancbmk_s-10B	0.2911	0.3136	0.3156	0.3217	0.3229	0.3230	**0.3230**
623.xalancbmk_s-165B	0.5531	0.5652	0.7183	0.7205	0.7207	**0.7207**	0.7207
625.x264_s-12B	1.5491	1.7205	1.7567	1.7761	1.7832	**1.7839**	1.7838
625.x264_s-18B	1.4318	1.5470	1.5738	1.6081	1.6085	1.6140	**1.6144**
627.cam4_s-490B	0.9434	1.0209	1.0305	1.0359	1.0326	1.0377	**1.0499**
627.cam4_s-573B	1.0529	1.1215	1.1426	1.2081	1.2272	1.2297	**1.2298**
628.pop2_s-17B	1.1552	1.1633	1.1668	1.2095	1.2285	1.2316	**1.2318**
631.deepsjeng_s-928B	0.9633	1.0510	1.0708	1.1778	1.1771	1.1904	**1.1922**
638.imagick_s-4128B	1.7865	1.7977	1.7977	1.8014	1.8677	**1.8689**	1.8689
638.imagick_s-824B	1.2097	1.2117	1.2100	1.2139	1.2139	**1.2140**	1.2139
641.leela_s-149B	0.7501	0.7652	0.7974	0.8702	0.8740	0.8848	**0.8850**
641.leela_s-334B	0.7381	0.7763	0.7975	0.9083	0.9134	0.9229	**0.9234**
641.leela_s-602B	0.7194	0.7422	0.7673	0.8485	0.8544	0.8638	**0.8641**
644.nab_s-5853B	1.1942	1.3427	1.3426	1.3430	1.3430	**1.3431**	1.3431
644.nab_s-7928B	1.2058	1.2077	1.2114	1.2122	1.2119	**1.2124**	1.2124
644.nab_s-9322B	1.0587	1.0606	1.0654	1.0667	1.0657	**1.0668**	1.0667
648.exchange2_s-1227B	1.2632	1.6124	1.7149	1.8981	1.9298	1.9494	**1.9499**
648.exchange2_s-1712B	1.3765	1.7710	1.9031	2.0936	2.1120	2.1211	**2.1212**
648.exchange2_s-210B	1.3171	1.7089	1.8339	2.0758	2.0884	2.1013	**2.1025**
648.exchange2_s-584B	1.2775	1.6481	1.7480	1.9857	2.0045	2.0182	**2.0202**
649.fotonik3d_s-1176B	**0.6074**	0.6074	0.6074	0.6074	0.6074	0.6074	0.6074
649.fotonik3d_s-1B	0.9909	1.1307	1.1641	1.2118	1.2214	**1.2220**	1.2220
649.fotonik3d_s-7084B	0.5978	0.5978	0.5978	**0.5978**	0.5978	0.5978	0.5978
654.roms_s-1021B	0.9792	1.2914	1.3217	1.4625	1.4670	**1.4787**	1.4787
654.roms_s-293B	0.9643	0.9644	0.9644	0.9643	0.9645	**0.9694**	0.9694
654.roms_s-842B	1.0134	1.5824	1.6398	1.7431	1.7507	1.7507	**1.7508**
657.xz_s-2302B	0.8823	0.9156	0.9283	0.9851	0.9769	0.9969	**0.9987**
657.xz_s-3167B	1.0146	1.0327	1.0420	1.1151	1.1181	1.1301	**1.1314**
657.xz_s-56B	1.6649	1.6679	1.7011	**1.7072**	1.7072	1.7072	1.7072

Table 12: MPKI of every predictor on every trace. Bim. = bimodal, Perc. = perceptron, Hashed P. = hashed perceptron, TSL = TAGE-SC-L, Proposed = TFL-TAGE-SC-L; bold = best on the trace.

Trace	Bim.	Gshare	Perc.	Hashed P.	TAGE	TSL	Proposed
600.perlbench_s-1273B	2.996	2.482	1.917	1.120	0.678	0.635	**0.634**
600.perlbench_s-210B	3.352	2.993	1.775	0.954	0.696	0.666	**0.659**
600.perlbench_s-570B	6.159	5.982	5.243	1.534	1.107	**1.051**	1.051
602.gcc_s-1850B	19.830	9.062	8.737	0.097	0.065	0.063	**0.032**
602.gcc_s-2226B	3.119	1.831	1.562	1.057	0.554	0.534	**0.149**
602.gcc_s-734B	12.790	0.899	0.855	0.235	0.197	0.155	**0.153**
603.bwaves_s-1080B	18.076	9.467	11.711	0.094	0.035	**0.014**	0.015
603.bwaves_s-2609B	4.685	0.787	0.007	0.007	0.007	**0.000**	0.000
603.bwaves_s-891B	24.103	0.223	0.223	0.223	0.223	0.000	**0.000**
605.mcf_s-1152B	21.058	17.734	15.850	13.981	14.537	13.920	**13.874**

Table 12: MPKI of every predictor on every trace. Bim. = bimodal, Perc. = perceptron, Hashed P. = hashed perceptron, TSL = TAGE-SC-L, Proposed = TFL-TAGE-SC-L; bold = best on the trace.

Trace	Bim.	Gshare	Perc.	Hashed P.	TAGE	TSL	Proposed
605.mcf_s-1536B	0.035	0.019	0.019	0.019	0.011	**0.011**	0.011
605.mcf_s-472B	1.826	1.682	1.596	1.106	1.117	1.033	**1.033**
607.cactuBSSN_s-2421B	0.013	0.009	0.007	0.007	0.005	**0.003**	0.003
607.cactuBSSN_s-3477B	0.003	0.001	0.002	0.001	0.001	**0.000**	0.000
619.lbm_s-2676B	0.211	0.116	0.114	0.109	0.101	**0.101**	0.101
619.lbm_s-3766B	0.632	0.402	0.373	0.366	0.374	**0.365**	0.367
620.omnetpp_s-141B	10.695	8.258	5.855	3.838	3.722	3.322	**3.278**
620.omnetpp_s-874B	8.732	6.953	4.975	3.727	3.665	3.237	**3.211**
621.wrf_s-575B	5.341	3.189	1.519	0.475	0.263	0.224	**0.223**
621.wrf_s-6673B	1.901	1.277	0.389	0.079	0.044	**0.026**	0.027
621.wrf_s-8100B	4.247	2.321	1.832	1.166	1.146	0.993	**0.918**
623.xalancbmk_s-10B	12.889	2.576	2.042	0.692	0.478	0.455	**0.453**
623.xalancbmk_s-165B	10.298	7.723	0.141	0.031	0.021	**0.020**	0.020
625.x264_s-12B	4.694	0.670	0.496	0.079	0.057	**0.053**	0.054
625.x264_s-18B	4.597	1.660	1.289	0.662	0.620	0.533	**0.527**
627.cam4_s-490B	11.935	5.741	5.101	4.696	4.904	4.578	**3.828**
627.cam4_s-573B	8.469	3.875	3.154	0.980	0.549	0.484	**0.478**
628.pop2_s-17B	3.713	3.258	3.065	1.197	0.450	0.333	**0.328**
631.deepsjeng_s-928B	13.746	9.760	8.481	4.166	4.194	3.761	**3.709**
638.imagick_s-4128B	1.707	0.889	0.889	0.842	0.023	**0.000**	0.000
638.imagick_s-824B	0.317	0.190	0.274	0.090	0.088	**0.088**	0.089
641.leela_s-149B	20.586	19.217	16.332	10.929	10.678	9.962	**9.936**
641.leela_s-334B	20.545	17.273	15.340	8.304	7.972	7.463	**7.425**
641.leela_s-602B	20.509	18.606	16.052	10.043	9.614	9.018	**8.986**
644.nab_s-5853B	2.804	0.028	0.033	0.015	0.012	**0.011**	0.011
644.nab_s-7928B	1.770	1.677	1.550	1.488	1.537	**1.484**	1.486
644.nab_s-9322B	2.486	2.387	2.205	2.118	2.190	**2.117**	2.120
648.exchange2_s-1227B	21.539	6.823	4.518	1.372	1.009	0.791	**0.782**
648.exchange2_s-1712B	19.997	5.838	3.366	0.721	0.498	0.391	**0.389**
648.exchange2_s-210B	22.410	6.879	4.366	0.822	0.668	0.520	**0.506**
648.exchange2_s-584B	22.621	7.332	5.034	1.228	0.998	0.823	**0.787**
649.fotonik3d_s-1176B	**0.004**	0.004	0.004	0.004	0.004	0.004	0.004
649.fotonik3d_s-1B	9.174	4.163	1.722	0.277	0.037	**0.001**	0.001
649.fotonik3d_s-7084B	0.056	0.056	0.056	0.000	**0.000**	0.000	0.000
654.roms_s-1021B	24.894	2.938	2.503	0.130	0.092	**0.001**	0.001
654.roms_s-293B	0.033	0.032	0.032	0.032	0.021	0.001	**0.001**
654.roms_s-842B	28.051	1.505	0.725	0.105	0.053	0.052	**0.052**
657.xz_s-2302B	15.390	13.153	11.979	9.417	9.705	8.950	**8.863**
657.xz_s-3167B	11.045	10.569	9.502	5.445	5.232	4.878	**4.813**
657.xz_s-56B	0.628	0.418	0.209	**0.000**	0.000	0.000	0.000

Table 13: Branch prediction accuracy (%, all branch types) of every predictor on every trace. Bim. = bimodal, Perc. = perceptron, Hashed P. = hashed perceptron, TSL = TAGE-SC-L, Proposed = TFL-TAGE-SC-L; bold = best on the trace.

Trace	Bim.	Gshare	Perc.	Hashed P.	TAGE	TSL	Proposed
600.perlbench_s-1273B	97.861	98.228	98.632	99.200	99.516	99.547	**99.548**
600.perlbench_s-210B	97.607	97.864	98.733	99.319	99.503	99.525	**99.530**
600.perlbench_s-570B	96.264	96.372	96.820	99.069	99.328	**99.363**	99.362
602.gcc_s-1850B	92.266	96.466	96.592	99.962	99.975	99.975	**99.988**
602.gcc_s-2226B	99.125	99.486	99.562	99.703	99.845	99.850	**99.958**
602.gcc_s-734B	94.674	99.626	99.644	99.902	99.918	99.935	**99.936**
603.bwaves_s-1080B	87.561	93.485	91.941	99.935	99.976	**99.990**	99.990
603.bwaves_s-2609B	81.232	96.848	99.971	99.971	99.971	**100.000**	100.000
603.bwaves_s-891B	85.619	99.867	99.867	99.867	99.867	100.000	**100.000**
605.mcf_s-1152B	91.852	93.138	93.867	94.590	94.375	94.614	**94.631**
605.mcf_s-1536B	99.978	99.988	99.988	99.988	99.993	**99.993**	99.993
605.mcf_s-472B	98.583	98.695	98.762	99.142	99.134	99.198	**99.199**

Table 13: Branch prediction accuracy (%, all branch types) of every predictor on every trace. Bim. = bimodal, Perc. = perceptron, Hashed P. = hashed perceptron, TSL = TAGE-SC-L, Proposed = TFL-TAGE-SC-L; bold = best on the trace.

Trace	Bim.	Gshare	Perc.	Hashed P.	TAGE	TSL	Proposed
607.cactuBSSN_s-2421B	99.039	99.356	99.483	99.530	99.662	**99.790**	99.787
607.cactuBSSN_s-3477B	99.962	99.986	99.982	99.987	99.985	**99.998**	99.997
619.lbm_s-2676B	98.257	99.043	99.055	99.097	99.162	**99.165**	99.163
619.lbm_s-3766B	95.949	97.427	97.612	97.656	97.604	**97.661**	97.647
620.omnetpp_s-141B	94.906	96.067	97.211	98.172	98.227	98.418	**98.439**
620.omnetpp_s-874B	95.802	96.658	97.609	98.208	98.238	98.444	**98.456**
621.wrf_s-575B	96.015	97.621	98.866	99.645	99.804	99.833	**99.833**
621.wrf_s-6673B	96.839	97.877	99.352	99.868	99.926	**99.957**	99.956
621.wrf_s-8100B	96.558	98.119	98.515	99.055	99.071	99.195	**99.256**
623.xalancbmk_s-10B	95.018	99.004	99.211	99.733	99.815	99.824	**99.825**
623.xalancbmk_s-165B	96.291	97.218	99.949	99.989	99.993	**99.993**	99.993
625.x264_s-12B	88.925	98.420	98.831	99.814	99.865	**99.875**	99.874
625.x264_s-18B	90.214	96.467	97.256	98.591	98.680	98.866	**98.879**
627.cam4_s-490B	87.572	94.022	94.688	95.110	94.893	95.233	**96.014**
627.cam4_s-573B	93.655	97.097	97.637	99.266	99.588	99.638	**99.642**
628.pop2_s-17B	97.565	97.863	97.989	99.215	99.705	99.781	**99.785**
631.deepsjeng_s-928B	90.656	93.365	94.235	97.168	97.149	97.443	**97.479**
638.imagick_s-4128B	99.118	99.541	99.541	99.565	99.988	**100.000**	100.000
638.imagick_s-824B	99.803	99.882	99.830	99.944	99.945	**99.945**	99.945
641.leela_s-149B	87.561	88.388	90.131	93.396	93.548	93.980	**93.996**
641.leela_s-334B	88.177	90.061	91.173	95.222	95.412	95.706	**95.727**
641.leela_s-602B	87.950	89.068	90.569	94.099	94.351	94.702	**94.720**
644.nab_s-5853B	97.103	99.971	99.966	99.984	99.987	**99.988**	99.988
644.nab_s-7928B	98.155	98.252	98.385	98.449	98.398	**98.453**	98.451
644.nab_s-9322B	98.069	98.146	98.287	98.355	98.299	**98.356**	98.354
648.exchange2_s-1227B	84.920	95.223	96.837	99.039	99.294	99.446	**99.453**
648.exchange2_s-1712B	85.089	95.647	97.490	99.463	99.628	99.709	**99.710**
648.exchange2_s-210B	83.549	94.950	96.795	99.396	99.509	99.618	**99.629**
648.exchange2_s-584B	83.921	94.788	96.422	99.127	99.290	99.415	**99.440**
649.fotonik3d_s-1176B	**99.975**	99.975	99.975	99.975	99.975	99.975	99.975
649.fotonik3d_s-1B	96.141	98.249	99.276	99.884	99.984	**99.999**	99.999
649.fotonik3d_s-7084B	99.186	99.186	99.186	99.998	**99.999**	99.999	99.999
654.roms_s-1021B	83.416	98.043	98.332	99.913	99.939	**99.999**	99.999
654.roms_s-293B	99.908	99.909	99.909	99.910	99.940	99.996	**99.996**
654.roms_s-842B	82.939	99.085	99.559	99.936	99.968	99.968	**99.968**
657.xz_s-2302B	87.690	89.479	90.419	92.468	92.237	92.841	**92.911**
657.xz_s-3167B	92.557	92.878	93.597	96.331	96.474	96.713	**96.757**
657.xz_s-56B	96.875	97.917	98.958	**100.000**	100.000	100.000	100.000

B Trace manifest

Table 14: Trace manifest: the 50 SPEC CPU2017 traces (Zenodo record 10960004); every file passed `xz -t` and matches Zenodo's size and MD5. Each trace holds 2,000M instructions.

Trace	Set	Suite	Size (MB)	MD5 (first 12 hex)
602.gcc_s-1850B	original 10	INT	704.8	`a01fb04176de`
605.mcf_s-1152B	original 10	INT	1,049.2	`b2a44b59bbc0`
607.cactuBSSN_s-2421B	original 10	FP	257.2	`8030545e6ef2`
619.lbm_s-2676B	original 10	FP	771.5	`840647f7f95b`
620.omnetpp_s-141B	original 10	INT	831.6	`6cdf4692ff2a`
623.xalancbmk_s-10B	original 10	INT	585.1	`3b16f6c9d358`
625.x264_s-12B	original 10	INT	247.4	`9fff2d0c19e2`
631.deepsjeng_s-928B	original 10	INT	266.3	`823e5fdcf525`
638.imagick_s-824B	original 10	FP	59.5	`fd4b5a944a45`
657.xz_s-2302B	original 10	INT	520.3	`4a5c15053d01`

Table 14: Trace manifest: the 50 SPEC CPU2017 traces (Zenodo record 10960004); every file passed `xz -t` and matches Zenodo's size and MD5. Each trace holds 2,000M instructions.

Trace	Set	Suite	Size (MB)	MD5 (first 12 hex)
600.perlbench_s-1273B	additional 40	INT	145.5	`eff9e488f6d5`
600.perlbench_s-210B	additional 40	INT	155.4	`8ac40f7348a2`
600.perlbench_s-570B	additional 40	INT	153.8	`1fb27aed6d81`
602.gcc_s-2226B	additional 40	INT	872.2	`b39ebc6bec73`
602.gcc_s-734B	additional 40	INT	368.7	`f3b5fb7301a2`
603.bwaves_s-1080B	additional 40	FP	39.6	`be58dacc1669`
603.bwaves_s-2609B	additional 40	FP	386.3	`14ac8ffc0a45`
603.bwaves_s-891B	additional 40	FP	413.2	`8efc36ccea12`
605.mcf_s-1536B	additional 40	INT	325.0	`73f39eb04b09`
605.mcf_s-472B	additional 40	INT	366.3	`692697643227`
607.cactuBSSN_s-3477B	additional 40	FP	228.2	`7d30808db6bc`
619.lbm_s-3766B	additional 40	FP	846.4	`800972ee67b5`
620.omnetpp_s-874B	additional 40	INT	802.3	`533f98aefe9d`
621.wrf_s-575B	additional 40	FP	352.8	`5ec5bd634b05`
621.wrf_s-6673B	additional 40	FP	282.0	`b3f103221b99`
621.wrf_s-8100B	additional 40	FP	275.0	`27305ea16679`
623.xalancbmk_s-165B	additional 40	INT	143.2	`dbc3cb4936d5`
625.x264_s-18B	additional 40	INT	246.5	`8c2f778998e9`
627.cam4_s-490B	additional 40	FP	394.3	`4c8247b1ccf2`
627.cam4_s-573B	additional 40	FP	151.9	`bb01996fa958`
628.pop2_s-17B	additional 40	FP	314.0	`db580bd8e9ca`
638.imagick_s-4128B	additional 40	FP	50.7	`ed67c3d3df78`
641.leela_s-149B	additional 40	INT	333.5	`934f4331c89d`
641.leela_s-334B	additional 40	INT	250.4	`e955331bff15`
641.leela_s-602B	additional 40	INT	307.3	`f593fe5bc590`
644.nab_s-5853B	additional 40	FP	123.5	`617ed8cc9f55`
644.nab_s-7928B	additional 40	FP	114.6	`f0835acc1b7b`
644.nab_s-9322B	additional 40	FP	134.9	`a6214cb70a92`
648.exchange2_s-1227B	additional 40	INT	70.3	`1f8aaf3702c3`
648.exchange2_s-1712B	additional 40	INT	50.5	`aef8988198e9`
648.exchange2_s-210B	additional 40	INT	49.6	`01e87200c2c8`
648.exchange2_s-584B	additional 40	INT	52.2	`4488cd35fe6b`
649.fotonik3d_s-1176B	additional 40	FP	328.7	`93afa8aeaab7`
649.fotonik3d_s-1B	additional 40	FP	25.3	`a9ee5d35387f`
649.fotonik3d_s-7084B	additional 40	FP	517.4	`d56355509b13`
654.roms_s-1021B	additional 40	FP	25.6	`0039e1b3316c`
654.roms_s-293B	additional 40	FP	315.2	`b18a970aee84`
654.roms_s-842B	additional 40	FP	199.9	`f60d65d0fd66`
657.xz_s-3167B	additional 40	INT	465.4	`cac895d9cebc`
657.xz_s-56B	additional 40	INT	93.8	`5211b1778f2c`

C Reproduction

The complete procedure is in `README.md`. In short (Linux or WSL, with `g++`, `xz`, `python3`):

```
# 1. traces: download the 50 files of experiments/traces/target_traces.tsv
#    from Zenodo record 10960004 into TRACE_DIR, then verify and map them
experiments/scripts/check_traces.sh TRACE_DIR experiments/traces/trace_integrity.txt
python3 experiments/scripts/reconcile_traces.py --trace-dir TRACE_DIR \
        --check experiments/traces/trace_integrity.txt   # -> trace_manifest.csv, trace_map.tsv
# 2. build the seven binaries
bash experiments/scripts/build_all_50trace.sh
# 3. run the 350 simulations (resumable)
python3 experiments/scripts/run_matrix.py --stage M50_50M_200M \
        --bins bimodal,gshare,perceptron,hashed_perceptron,tage,tsl_base,tsl_proposed \
```

```
       --trace-map experiments/traces/trace_map.tsv --warm 50 --sim 200 -j 14
# 4. analysis, figures, report values and PDF
python3 experiments/scripts/analyze_50trace.py
python3 experiments/scripts/make_figures_50trace.py      # needs matplotlib
python3 experiments/scripts/make_report_50trace.py
cd report_50trace && tectonic main.tex
```

The predictor alone can also be built with ChampSim's own script: `./build_champsim.sh tfl_tage_scl no no no no lru 1`.